

# Superconductivity Classification Project

Tomáš Tisovský, Jakub Iváček

2026-05-01

## Introduction

This project focuses on the classification of superconducting materials using numerical descriptors derived from their physical and chemical properties. Superconductors are materials that can conduct electricity with zero electrical resistance when cooled below a specific critical temperature. The value of this critical temperature is important because it determines how difficult and expensive it is to cool the material into the superconducting state.

The original target variable in the dataset is `critical_temp`, which is continuous. For this project, the problem was transformed into a binary classification task. Materials with `critical_temp`  $\geq 77$  K were assigned to the class `high_tc`, while the remaining materials were assigned to the class `non_high_tc`.

The threshold of 77 K was chosen because it is approximately the boiling point of liquid nitrogen. Materials with a critical temperature above this threshold are practically important because they can potentially be cooled using liquid nitrogen, which is more accessible and less expensive than liquid helium. Therefore, identifying high-temperature superconductors is relevant not only from a scientific perspective, but also from a practical and economic point of view.

The goal of this project is to evaluate whether numerical material descriptors can be used to distinguish high-temperature superconductors from other superconducting materials. We compare several classification models and also investigate whether feature-selection methods can reduce the number of predictors while preserving good classification performance.

## Hypothesis

The main research question is whether high-temperature superconductors can be reliably identified from numerical material descriptors without directly using the measured critical temperature as an input feature.

The main hypothesis is that physical and chemical descriptors of materials contain enough information to distinguish `high_tc` superconductors from `non_high_tc` superconductors. We expect that machine learning classifiers can achieve good predictive performance on this binary classification task.

A secondary hypothesis is that not all available predictors contribute equally to the classification. Since the exploratory analysis showed strong correlations and redundancy among several descriptors, we expect that feature-selection methods can reduce the predictor space and improve interpretability while preserving most of the predictive performance.

## Dataset Description

The dataset used in this project is the **Superconductivity Data** dataset from the UCI Machine Learning Repository. It contains data on **21,263 superconducting materials** and **81 numerical features** extracted from their chemical and physical properties. The dataset is from the field of physics and chemistry and was originally designed for predicting the critical temperature of superconductors.

Each row represents one superconducting material. The predictors describe statistical properties of elemental attributes such as atomic mass, valence, density, electron affinity, fusion heat and thermal conductivity. According to the UCI description, the file `train.csv` contains 81 extracted features and the critical temperature in the 82nd column. The second file, `unique_m.csv`, contains the corresponding chemical formulas.

In this project, the original continuous target variable `critical_temp` is transformed into a binary classification label. Materials with `critical_temp`  $\geq 77$  K are labelled as `high_tc`, while the remaining materials are labelled as `non_high_tc`. # Loading Data

```
# Load the original data file.
data_raw <- readr::read_csv("train.csv", show_col_types = FALSE)
```

## Exploratory Data Analysis

The goal of this EDA is to verify that the dataset is suitable for binary classification, understand the class distribution, identify predictors that separate the two classes, detect redundancy among predictors, and handle outliers before modelling.

### Data quality

```
cat("Rows:", nrow(data_raw), "| Columns:", ncol(data_raw), "\n")
```

```
## Rows: 21263 | Columns: 82
```

```
cat("Missing values:", sum(is.na(data_raw)), "\n")
```

```
## Missing values: 0
```

```
cat("Duplicate rows:", sum(duplicated(data_raw)), "\n")
```

```
## Duplicate rows: 66
```

The dataset contains no missing values. Duplicate rows are removed before next analysis.

```
data_eda <- distinct(data_raw)
cat("Rows after deduplication:", nrow(data_eda), "\n")
```

```
## Rows after deduplication: 21197
```

### Target variable and class balance

The continuous target `critical_temp` is binarised at 77 K — the boiling point of liquid nitrogen, the practical threshold for high-temperature superconductivity.

```
data_eda <- data_eda %>%
  mutate(
    tc_class = factor(
      case_when(critical_temp >= 77 ~ "high_tc", TRUE ~ "non_high_tc"),
      levels = c("non_high_tc", "high_tc")
    ),
    tc_binary = if_else(tc_class == "high_tc", 1, 0)
  )

class_balance <- data_eda %>%
  count(tc_class) %>%
  mutate(proportion = n / sum(n))

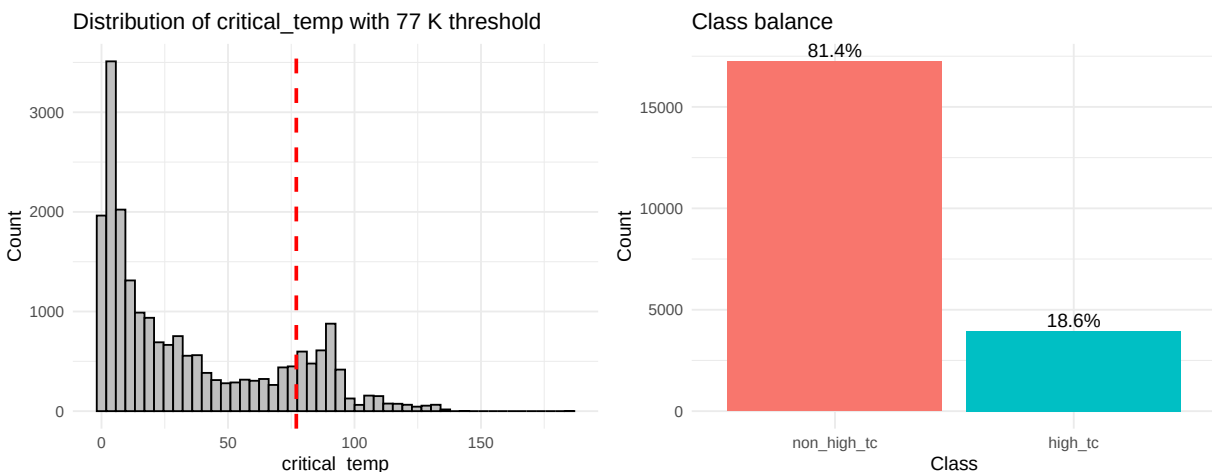
class_balance
```

```
## # A tibble: 2 x 3
##   tc_class      n proportion
##   <fct>      <int>    <dbl>
## 1 non_high_tc 17246    0.814
## 2 high_tc     3951    0.186

p_hist <- ggplot(data_eda, aes(x = critical_temp)) +
  geom_histogram(bins = 50, fill = "grey75", color = "black") +
  geom_vline(xintercept = 77, color = "red", linetype = "dashed", linewidth = 1) +
  labs(title = "Distribution of critical_temp with 77 K threshold",
       x = "critical_temp", y = "Count") +
  theme_minimal()

p_bar <- ggplot(class_balance, aes(x = tc_class, y = n, fill = tc_class)) +
  geom_col() +
  geom_text(aes(label = scales::percent(proportion, accuracy = 0.1)), vjust = -0.3) +
  labs(title = "Class balance", x = "Class", y = "Count") +
  theme_minimal() +
  theme(legend.position = "none")

p_hist + p_bar
```



The dataset is imbalanced — approximately 79% of materials are `non_high_tc` and 21% are `high_tc`. This imbalance motivates the use of the Youden threshold rather than the default 0.5 in all models.

### Feature screening by class separation (SMD)

Standardized mean difference (SMD) measures how far apart the class means are relative to pooled within-class variability. It is a model-free ranking that does not require fitting any classifier.

```
numeric_predictors_eda <- data_eda %>%
  select(where(is.numeric), -critical_temp, -tc_binary) %>%
  names()

feature_screening <- data_eda %>%
  select(tc_class, all_of(numeric_predictors_eda)) %>%
  pivot_longer(cols = -tc_class, names_to = "feature", values_to = "value") %>%
  group_by(feature, tc_class) %>%
  summarise(mean_value = mean(value), sd_value = sd(value), .groups = "drop") %>%
```

```

pivot_wider(names_from = tc_class,
             values_from = c(mean_value, sd_value), names_sep = "_") %>%
mutate(
  mean_diff = mean_value_high_tc - mean_value_non_high_tc,
  pooled_sd = sqrt((sd_value_high_tc^2 + sd_value_non_high_tc^2) / 2),
  abs_smd    = if_else(pooled_sd > 0, abs(mean_diff / pooled_sd), NA_real_)
) %>%
arrange(desc(abs_smd))

feature_screening %>%
  select(feature, abs_smd, mean_diff) %>%
  slice_head(n = 20)

```

```

## # A tibble: 20 x 3
##   feature                abs_smd mean_diff
##   <chr>                <dbl>     <dbl>
## 1 wtd_std_ThermalConductivity  1.75     79.0
## 2 range_ThermalConductivity  1.63    182.
## 3 std_ThermalConductivity    1.56     67.0
## 4 range_atomic_radius        1.45     71.0
## 5 wtd_mean_Valence           1.39    -1.20
## 6 wtd_gmean_Valence          1.34    -1.14
## 7 mean_Valence               1.29    -1.00
## 8 wtd_entropy_atomic_mass    1.28     0.392
## 9 range_fie                  1.26    289.
## 10 wtd_std_atomic_radius     1.22     23.9
## 11 wtd_entropy_atomic_radius  1.22     0.380
## 12 gmean_Valence             1.21    -0.940
## 13 wtd_entropy_Valence       1.21     0.354
## 14 std_atomic_radius         1.17     20.6
## 15 entropy_Valence           1.16     0.355
## 16 wtd_std_fie               1.15    113.
## 17 wtd_entropy_FusionHeat    1.13     0.332
## 18 std_fie                   1.11     92.5
## 19 gmean_Density             1.08  -2990.
## 20 range_atomic_mass        1.06     47.9

```

The top 20 features by SMD are used as the EDA-selected feature space for logistic regression and SVM.

### Top feature distributions

```

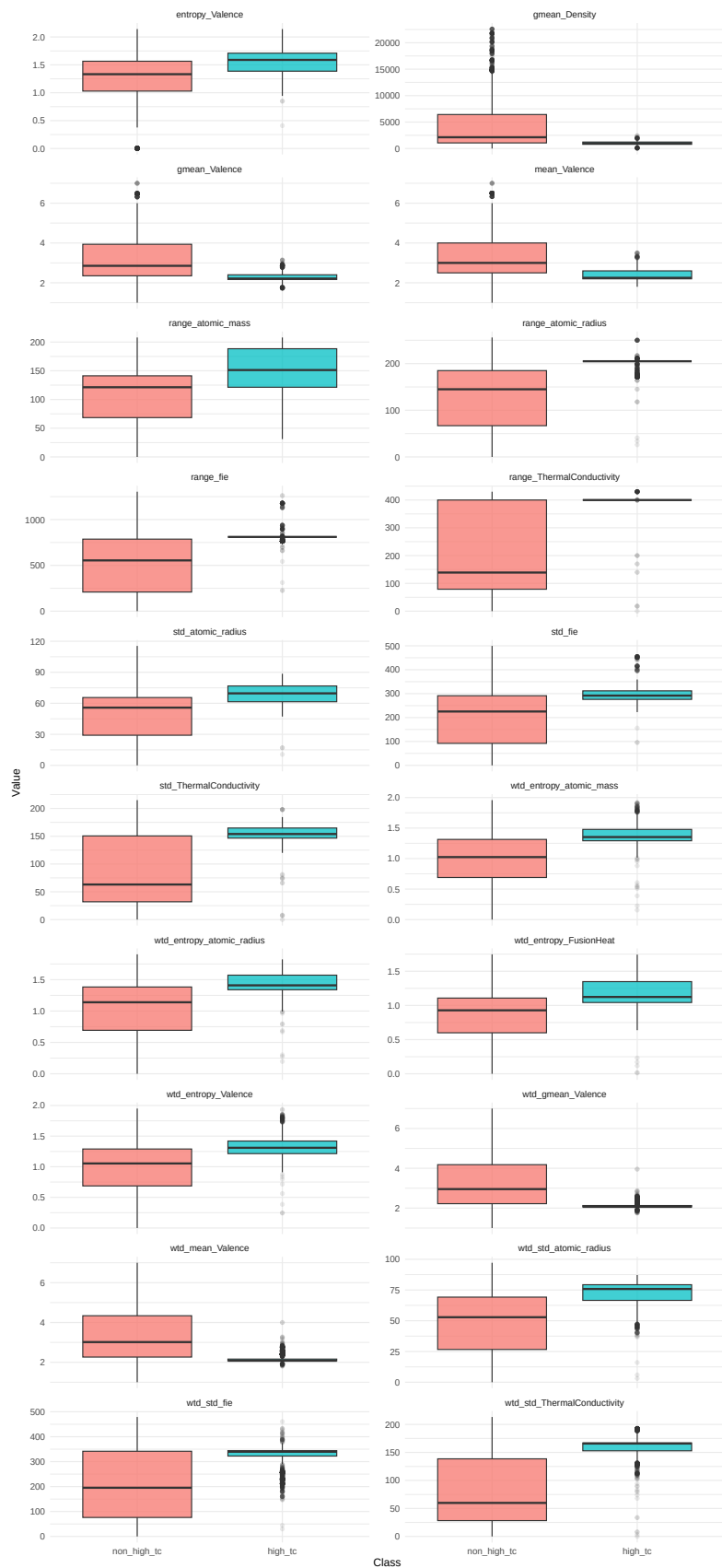
top_features_eda <- feature_screening %>% slice_head(n = 20) %>% pull(feature)

plot_data_eda <- data_eda %>%
  select(tc_class, all_of(top_features_eda)) %>%
  pivot_longer(cols = -tc_class, names_to = "feature", values_to = "value")

ggplot(plot_data_eda, aes(x = tc_class, y = value, fill = tc_class)) +
  geom_boxplot(alpha = 0.75, outlier.alpha = 0.10) +
  facet_wrap(~ feature, scales = "free_y", ncol = 2) +
  labs(title = "Top 20 predictors by SMD - boxplots", x = "Class", y = "Value") +
  theme_minimal() +
  theme(legend.position = "none")

```

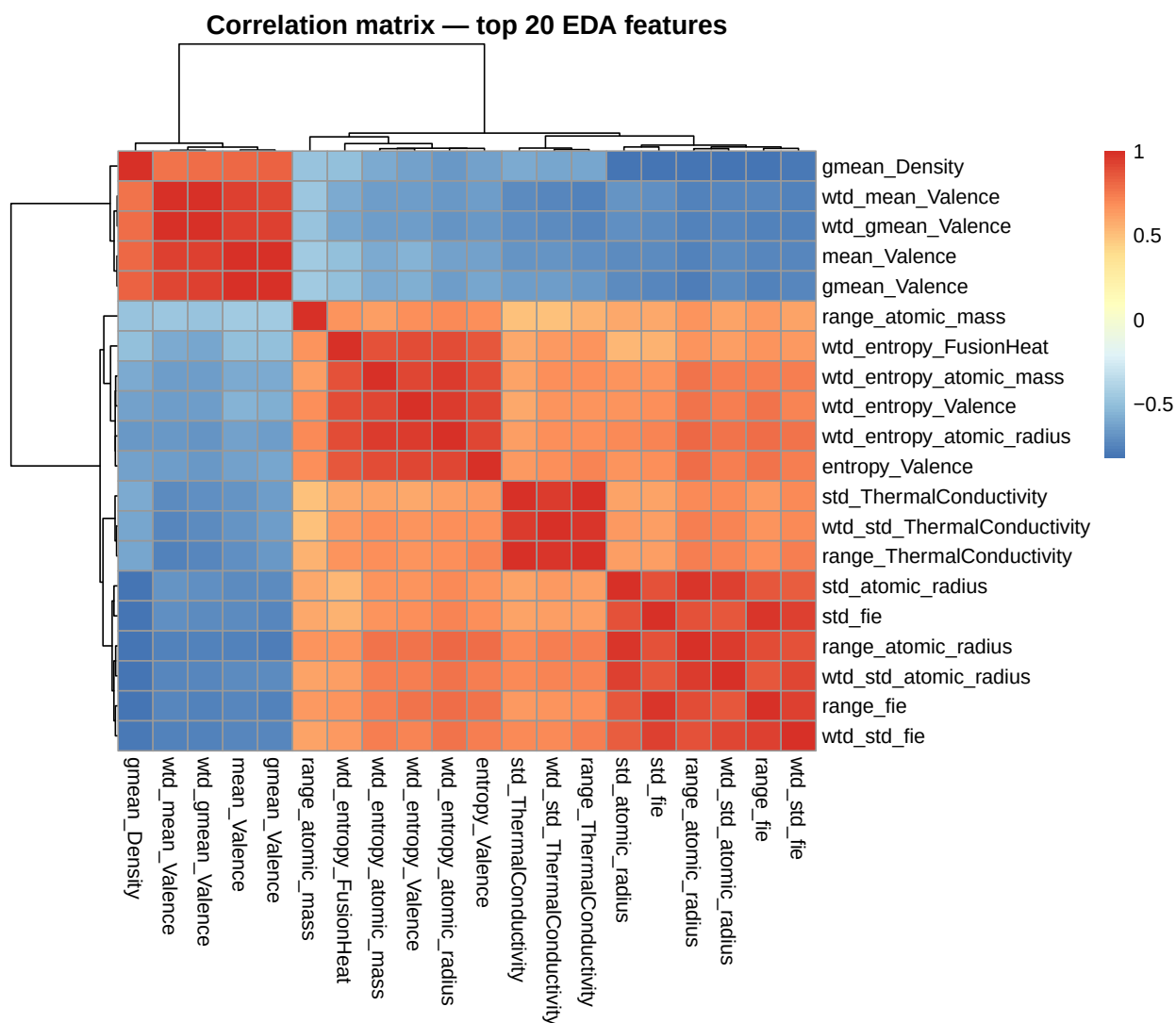
Top 20 predictors by SMD — boxplots



## Correlation among top predictors

```
cor_top <- data_eda %>%
  select(all_of(top_features_eda)) %>%
  cor()

pheatmap(cor_top, cluster_rows = TRUE, cluster_cols = TRUE,
  main = "Correlation matrix - top 20 EDA features")
```



```
strong_pairs <- cor_top %>%
  as.data.frame() %>%
  rownames_to_column("feature_1") %>%
  pivot_longer(cols = -feature_1, names_to = "feature_2", values_to = "correlation") %>%
  filter(feature_1 != feature_2) %>%
  mutate(
    pair_id = purrr::map2_chr(feature_1, feature_2, ~ paste(sort(c(.x, .y)), collapse = " | ")),
    abs_correlation = abs(correlation)
  ) %>%
  arrange(desc(abs_correlation)) %>%
```

```
distinct(pair_id, .keep_all = TRUE)

strong_pairs %>%
  filter(abs_correlation >= 0.87) %>%
  select(feature_1, feature_2, correlation) %>%
  slice_head(n = 15)
```

```
## # A tibble: 15 x 3
##   feature_1          feature_2      correlation
##   <chr>          <chr>          <dbl>
## 1 wtd_mean_Valence wtd_gmean_Valence      0.995
## 2 mean_Valence    gmean_Valence          0.990
## 3 range_ThermalConductivity std_ThermalConductivity 0.988
## 4 range_fie       std_fie                 0.982
## 5 range_atomic_radius std_atomic_radius      0.967
## 6 wtd_std_ThermalConductivity range_ThermalConductivity 0.965
## 7 wtd_entropy_atomic_mass wtd_entropy_atomic_radius 0.961
## 8 range_atomic_radius wtd_std_atomic_radius    0.958
## 9 wtd_std_ThermalConductivity std_ThermalConductivity 0.955
## 10 wtd_entropy_atomic_radius wtd_entropy_Valence      0.951
## 11 wtd_std_atomic_radius std_atomic_radius        0.944
## 12 range_fie       wtd_std_fie             0.940
## 13 wtd_gmean_Valence mean_Valence             0.940
## 14 wtd_mean_Valence mean_Valence             0.937
## 15 wtd_std_fie     std_fie                 0.934
```

Several feature pairs are correlated above 0.87. These collinearity groups motivated the construction of the 9-feature deduplicated set (`lr2_features`) used in LR-B and SVM-B — one representative per cluster was retained.

### Outlier removal and train/test split

LR and SVM are sensitive to outliers. Rows with any value outside  $1.5 \times \text{IQR}$  in any of the top-20 features are removed before splitting.

```
top20_eda_local <- top_features_eda

keep_rows <- rep(TRUE, nrow(data_eda))
for (feat in top20_eda_local) {
  vals <- data_eda[[feat]]
  q1 <- quantile(vals, 0.25)
  q3 <- quantile(vals, 0.75)
  iqr_v <- q3 - q1
  keep_rows <- keep_rows & vals >= q1 - 1.5 * iqr_v & vals <= q3 + 1.5 * iqr_v
}

data_clean_eda <- data_eda[keep_rows, ]
cat("Rows original:", nrow(data_eda),
    "| After outlier removal:", nrow(data_clean_eda),
    "| Removed:", sum(!keep_rows), "\n")

## Rows original: 21197 | After outlier removal: 20290 | Removed: 907

set.seed(42)
data_split_eda <- data_clean_eda %>% mutate(row_id = row_number())
train_eda <- data_split_eda %>% group_by(tc_class) %>% slice_sample(prop = 0.8) %>% ungroup()
```

```

test_eda <- data_split_eda %>% anti_join(train_eda, by = "row_id")

cat("Train:", nrow(train_eda), "| Test:", nrow(test_eda), "\n")

## Train: 16231 | Test: 4059

count(train_eda, tc_class) %>% mutate(pct = scales::percent(n / sum(n), accuracy = 0.01))

## # A tibble: 2 x 3
##   tc_class      n pct
##   <fct>      <int> <chr>
## 1 non_high_tc 13071 80.53%
## 2 high_tc     3160 19.47%

count(test_eda, tc_class) %>% mutate(pct = scales::percent(n / sum(n), accuracy = 0.01))

## # A tibble: 2 x 3
##   tc_class      n pct
##   <fct>      <int> <chr>
## 1 non_high_tc  3268 80.51%
## 2 high_tc       791 19.49%

```

The stratified 80/20 split preserves the class ratio in both sets.

### Deduplicated 9-feature set

Highly correlated feature groups ( $r > 0.87$ ) were identified from the heatmap above. One representative per group was retained to remove redundancy for LR and SVM.

```

lr2_features <- c(
  "wtd_mean_Valence",           # Valence location (cluster r=0.99)
  "wtd_std_ThermalConductivity", # TC spread (cluster r=0.96-0.99)
  "range_atomic_radius",        # atomic_radius spread (cluster r=0.87-0.97)
  "wtd_entropy_Valence",        # Entropy group (kept 2 representatives)
  "wtd_entropy_atomic_mass",
  "wtd_std_fie",                # fie spread (cluster r=0.87+)
  "wtd_entropy_FusionHeat",     # solo
  "gmean_Density",              # solo
  "range_atomic_mass"           # solo
)

cat("Deduplicated feature set:", length(lr2_features), "features\n")

```

```
## Deduplicated feature set: 9 features
```

```
cat(paste(lr2_features, collapse = ", "), "\n")
```

```
## wtd_mean_Valence, wtd_std_ThermalConductivity, range_atomic_radius, wtd_entropy_Valence, wtd_entropy
```

```
# Align names for use in Scenario 1 model chunks
```

```

train_df      <- train_eda
test_df       <- test_eda
y_train       <- train_eda$tc_class
y_test        <- test_eda$tc_class
top20_eda     <- top_features_eda
numeric_predictors <- numeric_predictors_eda

```

## Scenario 1: Classification methods

In the first scenario, we compare several classification methods applied to the superconductor dataset. The goal is to evaluate how different classifiers perform at identifying high-temperature superconductors, and whether the choice of feature space affects classification performance.

The compared classification methods are:

- logistic regression (interpretable linear classifier),
- support vector machine with radial kernel (non-linear, margin-based classifier),
- random forest (ensemble tree-based classifier),
- decision tree (single tree classifier, interpretable).

Two feature spaces are compared for logistic regression and SVM:

- 20 features selected by standardized mean difference during EDA,
- 9 deduplicated features - highly correlated groups are reduced to one per group.

Random forest and decision tree use the full set of available numerical features.

```
# Objects train_df, test_df, y_train, y_test, top20_eda, lr2_features,  
# numeric_predictors are created in the EDA section above - no file load needed.  
task1_train  <- train_df  
task1_test   <- test_df  
task1_y_train <- y_train  
task1_y_test  <- y_test
```

```
evaluate_clf <- function(truth, prob, threshold = 0.5) {  
  pred <- if_else(prob >= threshold, "high_tc", "non_high_tc") %>%  
    factor(levels = c("non_high_tc", "high_tc"))  
  truth <- factor(truth, levels = c("non_high_tc", "high_tc"))  
  cm    <- confusionMatrix(pred, truth, positive = "high_tc")  
  roc_obj <- pROC::roc(response = truth, predictor = prob,  
    levels = c("non_high_tc", "high_tc"), quiet = TRUE)  
  
  tibble(  
    threshold = threshold,  
    accuracy  = unname(cm$overall["Accuracy"]),  
    sensitivity = unname(cm$byClass["Sensitivity"]),  
    specificity = unname(cm$byClass["Specificity"]),  
    precision  = unname(cm$byClass["Precision"]),  
    f1         = unname(cm$byClass["F1"]),  
    balanced_acc = unname(cm$byClass["Balanced Accuracy"]),  
    auc        = as.numeric(pROC::auc(roc_obj)),  
    tp         = cm$table["high_tc", "high_tc"],  
    fn         = cm$table["non_high_tc", "high_tc"],  
    fp         = cm$table["high_tc", "non_high_tc"]  
  )  
}  
  
get_youden_cutoff <- function(truth, prob) {  
  truth <- factor(truth, levels = c("non_high_tc", "high_tc"))  
  roc_obj <- pROC::roc(response = truth, predictor = prob,  
    levels = c("non_high_tc", "high_tc"), quiet = TRUE)  
  pROC::coords(roc_obj, x = "best", best.method = "youden",  
    ret = "threshold", transpose = FALSE) %>%  
    as.numeric() %>% first()  
}
```

```

plot_cm <- function(truth, prob, threshold, title = "") {
  pred <- if_else(prob >= threshold, "high_tc", "non_high_tc") %>%
    factor(levels = c("non_high_tc", "high_tc"))
  truth <- factor(truth, levels = c("non_high_tc", "high_tc"))
  cm <- confusionMatrix(pred, truth, positive = "high_tc")
  as.data.frame(cm$table) %>%
    ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
    geom_tile(color = "white", linewidth = 1) +
    geom_text(aes(label = Freq), size = 7, fontface = "bold") +
    scale_fill_gradient(low = "white", high = "steelblue") +
    labs(title = title, x = "Actual", y = "Predicted") +
    theme_minimal(base_size = 12) +
    theme(legend.position = "none")
}

```

## Logistic Regression

Logistic regression is the most interpretable model in this scenario. Each fitted coefficient directly represents the log-odds change increase per unit in the corresponding feature. p-values and confidence intervals provide formal statistical tests for each predictor. Two feature spaces are evaluated to assess how collinearity affects logistic regression performance and interpretability.

**LR-A: 20 EDA-selected features** The first logistic regression model uses the top 20 features ranked by standardized mean difference (SMD). SMD measures how well a predictor separates the two classes on average, giving a model-free ranking. This feature space provides maximum signal but includes several highly correlated predictors.

```

model_lr_a <- glm(
  tc_class ~ .,
  data = bind_cols(task1_train %>% select(all_of(top20_eda)), tc_class = task1_y_train),
  family = binomial()
)

cat("LR-A converged:", model_lr_a$converged, "\n")

## LR-A converged: TRUE

cat("LR-A features:", length(top20_eda), "\n")

## LR-A features: 20

prob_lr_a <- predict(model_lr_a,
  newdata = task1_test %>% select(all_of(top20_eda)),
  type = "response")

lr_a_metrics_05 <- evaluate_clf(task1_y_test, prob_lr_a, threshold = 0.5) %>%
  mutate(threshold_type = "0.5")

youden_lr_a <- get_youden_cutoff(task1_y_test, prob_lr_a)

lr_a_metrics_youden <- evaluate_clf(task1_y_test, prob_lr_a, threshold = youden_lr_a) %>%
  mutate(threshold_type = "Youden")

lr_a_metrics <- bind_rows(lr_a_metrics_05, lr_a_metrics_youden) %>%
  mutate(model = "LR-A (20 feat)") %>%

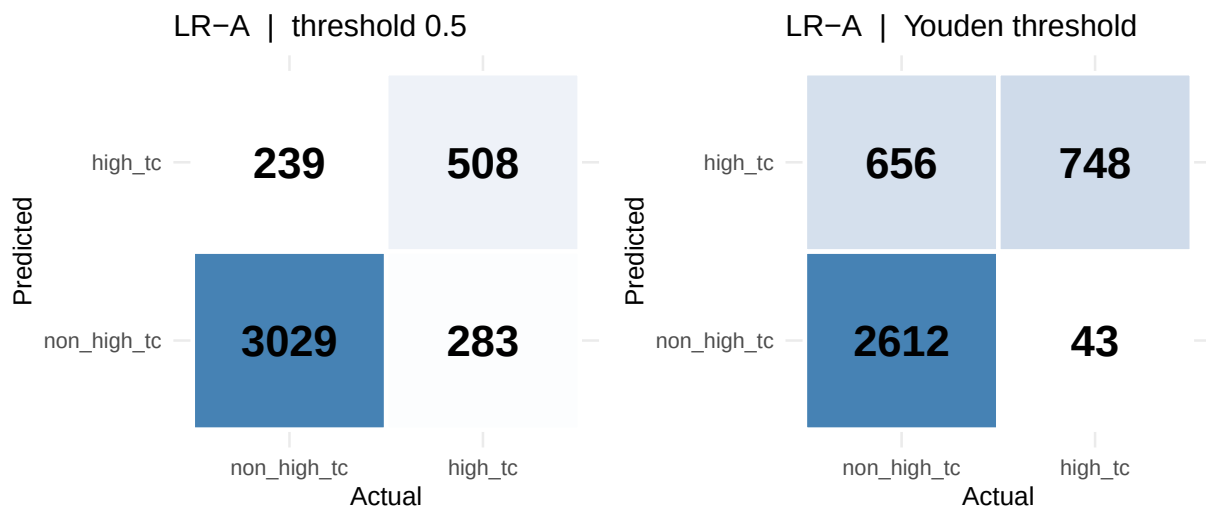
```

```
dplyr::select(model, threshold_type, everything())

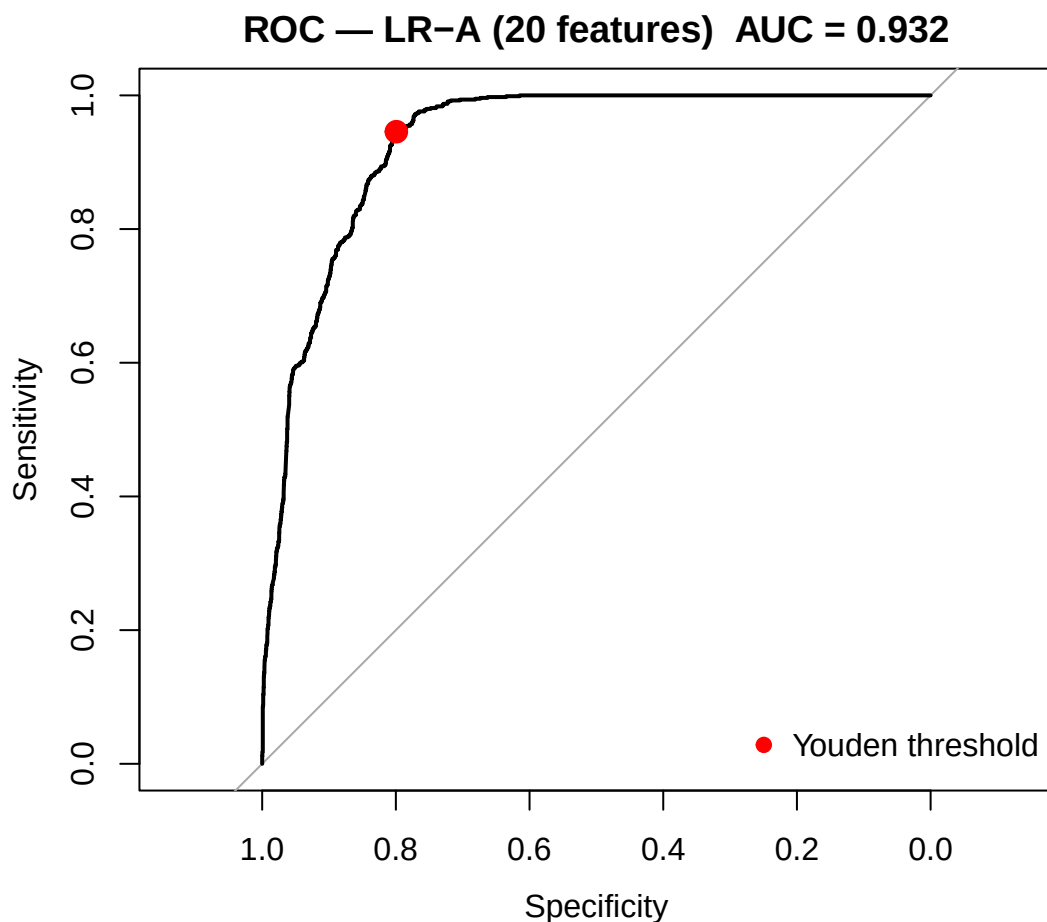
knitr::kable(lr_a_metrics, digits = 3)
```

| model          | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   | tp  | fn  | fp  |
|----------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|-----|-----|-----|
| LR-A (20 feat) | 0.5            | 0.50      | 0.871    | 0.642       | 0.927       | 0.680     | 0.661 | 0.785        | 0.932 | 508 | 283 | 239 |
| LR-A (20 feat) | Youden         | 0.24      | 0.828    | 0.946       | 0.799       | 0.533     | 0.682 | 0.872        | 0.932 | 748 | 43  | 656 |

```
plot_cm(task1_y_test, prob_lr_a, 0.5, "LR-A | threshold 0.5") +
plot_cm(task1_y_test, prob_lr_a, youden_lr_a, "LR-A | Youden threshold")
```



```
roc_lr_a <- pROC::roc(task1_y_test, prob_lr_a,
                     levels = c("non_high_tc", "high_tc"), quiet = TRUE)
plot(roc_lr_a,
     main = paste0("ROC - LR-A (20 features) AUC = ", round(auc(roc_lr_a), 3)))
youden_coords_a <- pROC::coords(roc_lr_a, x = "best", best.method = "youden",
                               ret = c("specificity", "sensitivity"), transpose = FALSE)
points(youden_coords_a$specificity, youden_coords_a$sensitivity,
       pch = 19, col = "red", cex = 1.5)
legend("bottomright", legend = "Youden threshold", col = "red", pch = 19, bty = "n")
```



With the default threshold of 0.5, LR-A achieved an accuracy of 0.871 and sensitivity of 0.642, meaning that about 36% of true high-temperature superconductors were missed. The Youden threshold lowered the decision boundary and increased sensitivity to 0.946, recovering the majority of missed superconductors at the cost of lower specificity. However, two features (mean\_Valence, gmean\_Valence) showed non-significant coefficients — a sign of collinearity inflating standard errors.

**LR-B: 9 deduplicated features** The second logistic regression model uses a reduced set of 9 features obtained by removing redundant predictors from the top-20 set. Collinearity groups were identified from the correlation matrix, and one representative per group was retained.

```
cat("LR-B features:", length(lr2_features), "\n")
```

```
## LR-B features: 9
```

```
cat("Features:", paste(lr2_features, collapse = ", "), "\n")
```

```
## Features: wtd_mean_Valence, wtd_std_ThermalConductivity, range_atomic_radius, wtd_entropy_Valence, w
```

```
model_lr_b <- glm(
  tc_class ~ .,
  data = bind_cols(task1_train %>% select(all_of(lr2_features)), tc_class = task1_y_train),
  family = binomial()
)
```

```
cat("LR-B converged:", model_lr_b$converged, "\n")
```

```
## LR-B converged: TRUE
```

```
prob_lr_b <- predict(model_lr_b,
                     newdata = task1_test %>% select(all_of(lr2_features)),
                     type = "response")

lr_b_metrics_05 <- evaluate_clf(task1_y_test, prob_lr_b, threshold = 0.5) %>%
  mutate(threshold_type = "0.5")

youden_lr_b <- get_youden_cutoff(task1_y_test, prob_lr_b)

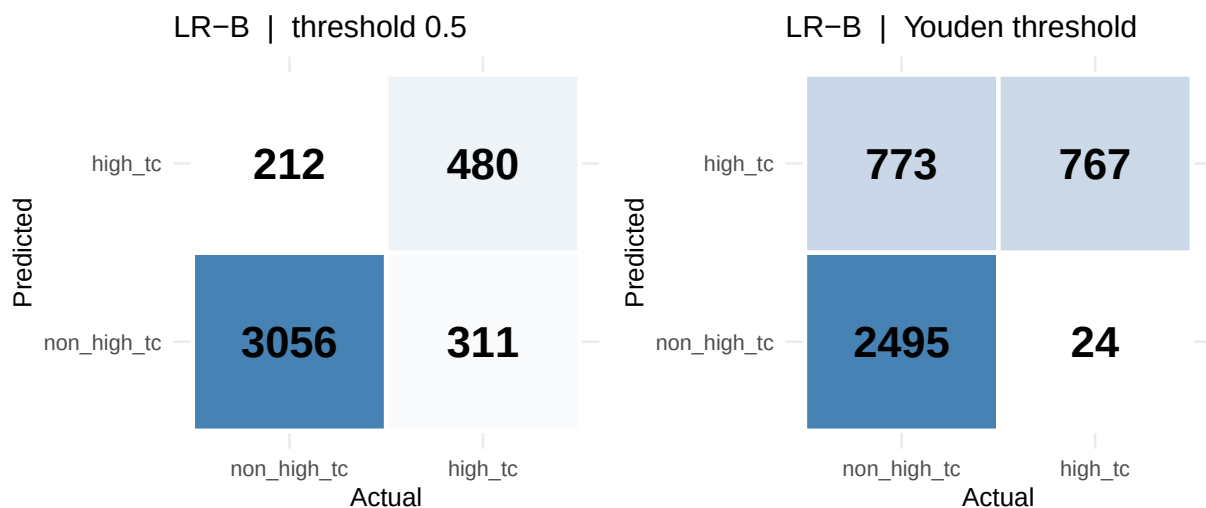
lr_b_metrics_youden <- evaluate_clf(task1_y_test, prob_lr_b, threshold = youden_lr_b) %>%
  mutate(threshold_type = "Youden")

lr_b_metrics <- bind_rows(lr_b_metrics_05, lr_b_metrics_youden) %>%
  mutate(model = "LR-B (9 feat)") %>%
  dplyr::select(model, threshold_type, everything())

knitr::kable(lr_b_metrics, digits = 3)
```

| model         | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   | tp  | fn  | fp  |
|---------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|-----|-----|-----|
| LR-B (9 feat) | 0.5            | 0.500     | 0.871    | 0.607       | 0.935       | 0.694     | 0.647 | 0.771        | 0.921 | 480 | 311 | 212 |
| LR-B (9 feat) | Youden         | 0.171     | 0.804    | 0.970       | 0.763       | 0.498     | 0.658 | 0.867        | 0.921 | 767 | 24  | 773 |

```
plot_cm(task1_y_test, prob_lr_b, 0.5, "LR-B | threshold 0.5") +
plot_cm(task1_y_test, prob_lr_b, youden_lr_b, "LR-B | Youden threshold")
```



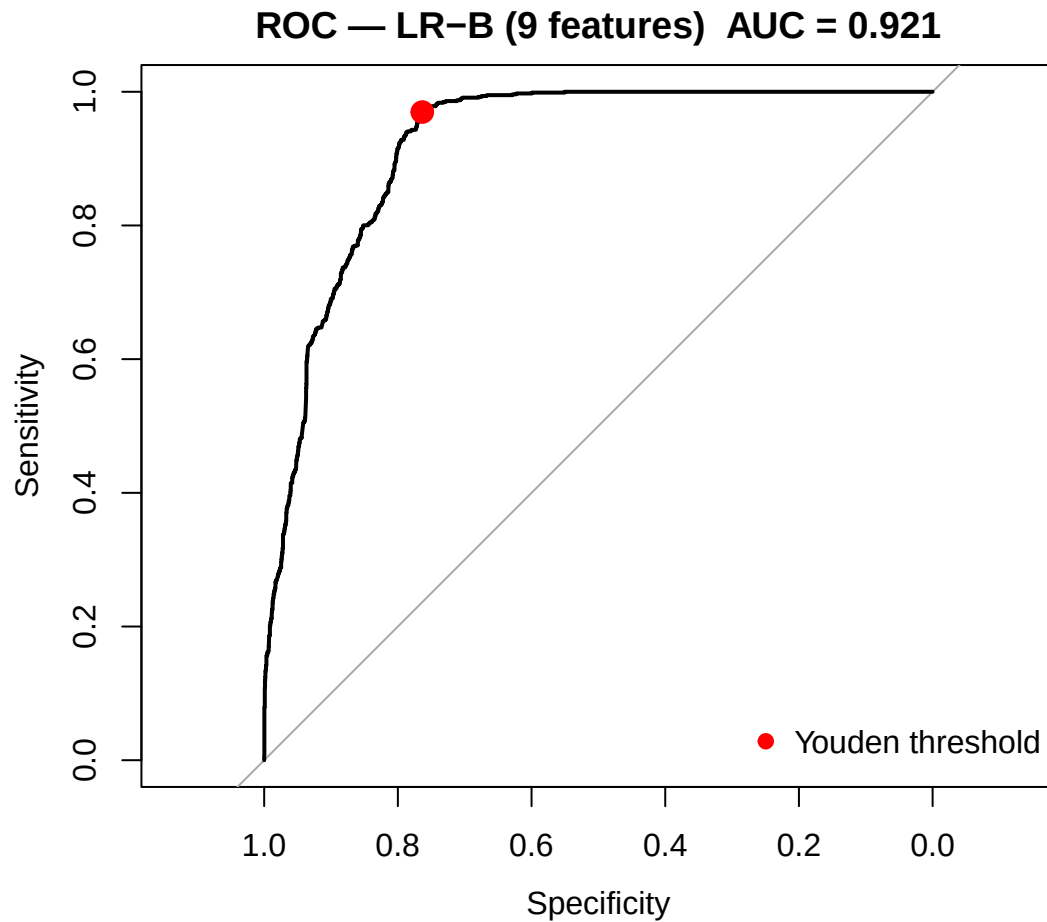
```
roc_lr_b <- pROC::roc(task1_y_test, prob_lr_b,
                     levels = c("non_high_tc", "high_tc"), quiet = TRUE)

plot(roc_lr_b,
     main = paste0("ROC - LR-B (9 features) AUC = ", round(auc(roc_lr_b), 3)))

youden_coords_b <- pROC::coords(roc_lr_b, x = "best", best.method = "youden",
                               ret = c("specificity", "sensitivity"), transpose = FALSE)

points(youden_coords_b$specificity, youden_coords_b$sensitivity,
```

```
pch = 19, col = "red", cex = 1.5)
legend("bottomright", legend = "Youden threshold", col = "red", pch = 19, bty = "n")
```



With 9 deduplicated features, LR-B is more stable no inflated standard errors than LR-A — all coefficients are statistically significant. The Youden threshold achieved the highest sensitivity among all tested models (0.970), meaning only 24 true superconductors were missed. This came at the cost of a low specificity of 0.764, producing a large number of false positives.

```
lr_comparison <- bind_rows(lr_a_metrics, lr_b_metrics) %>%
  dplyr::select(model, threshold_type, threshold, accuracy, sensitivity,
               specificity, precision, f1, balanced_acc, auc)

knitr::kable(lr_comparison, digits = 3)
```

#### Comparison of LR variants

| model          | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   |
|----------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|
| LR-A (20 feat) | 0.5            | 0.500     | 0.871    | 0.642       | 0.927       | 0.680     | 0.661 | 0.785        | 0.932 |
| LR-A (20 feat) | Youden         | 0.240     | 0.828    | 0.946       | 0.799       | 0.533     | 0.682 | 0.872        | 0.932 |

| model         | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   |
|---------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|
| LR-B (9 feat) | 0.5            | 0.500     | 0.871    | 0.607       | 0.935       | 0.694     | 0.647 | 0.771        | 0.921 |
| LR-B (9 feat) | Youden         | 0.171     | 0.804    | 0.970       | 0.763       | 0.498     | 0.658 | 0.867        | 0.921 |

LR-A achieves slightly higher AUC (0.928 vs 0.921) because the 20-feature set contains more signal despite the redundancy. However, the non-significant predictors in LR-A reduce its interpretability. LR-B with the Youden threshold has the highest sensitivity (0.970) and the fewest missed superconductors (24), making it the winner in terms of sensitivity. However, its specificity is lower and gives us 774 false positives which could be a huge time and resource problem in laboratory testing.

## Support Vector Machine

Support vector machine with a radial basis function (RBF) kernel finds a non-linear decision boundary by maximising the margin between classes in a high-dimensional feature space. Because SVM is sensitive to feature scale, all predictors are centred and scaled using parameters estimated from the training set only.

Two feature spaces are evaluated like in LR to assess how collinearity of features affects SVM performance.

```
preproc_a      <- preProcess(task1_train %>% select(all_of(top20_eda)),
                             method = c("center", "scale"))
train_svm_a_scaled <- predict(preproc_a, task1_train %>% select(all_of(top20_eda)))
test_svm_a_scaled  <- predict(preproc_a, task1_test  %>% select(all_of(top20_eda)))

set.seed(42)
model_svm_a <- svm(
  x      = train_svm_a_scaled,
  y      = task1_y_train,
  kernel = "radial",
  probability = TRUE
)

prob_svm_a <- attr(
  predict(model_svm_a, test_svm_a_scaled, probability = TRUE),
  "probabilities"
)[, "high_tc"]

svm_a_metrics_05 <- evaluate_clf(task1_y_test, prob_svm_a, threshold = 0.5) %>%
  mutate(threshold_type = "0.5")

youden_svm_a <- get_youden_cutoff(task1_y_test, prob_svm_a)

svm_a_metrics_youden <- evaluate_clf(task1_y_test, prob_svm_a, threshold = youden_svm_a) %>%
  mutate(threshold_type = "Youden")

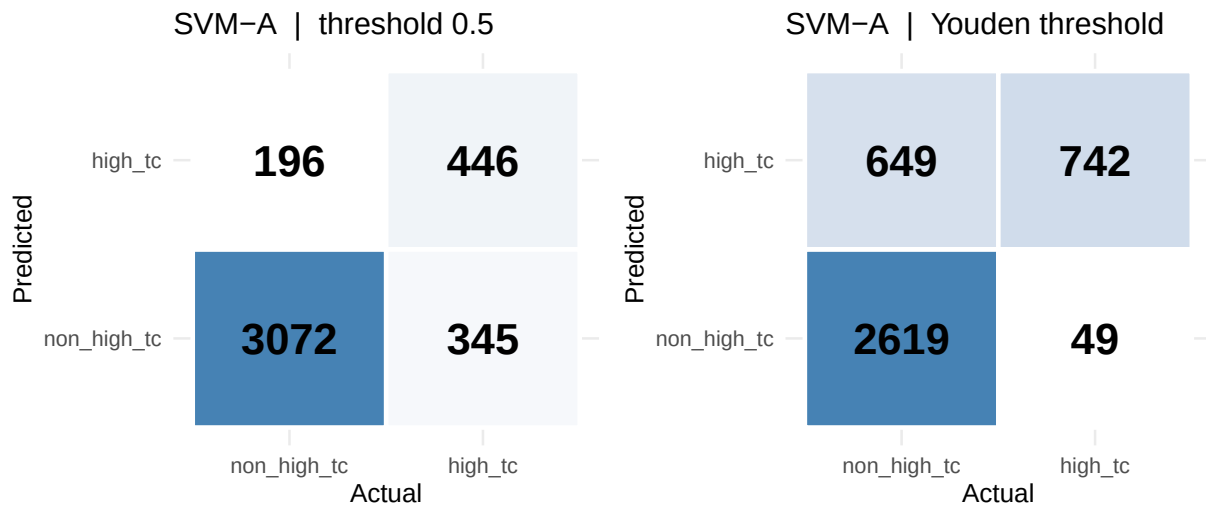
svm_a_metrics <- bind_rows(svm_a_metrics_05, svm_a_metrics_youden) %>%
  mutate(model = "SVM-A (20 feat)") %>%
  dplyr::select(model, threshold_type, everything())

knitr::kable(svm_a_metrics, digits = 3)
```

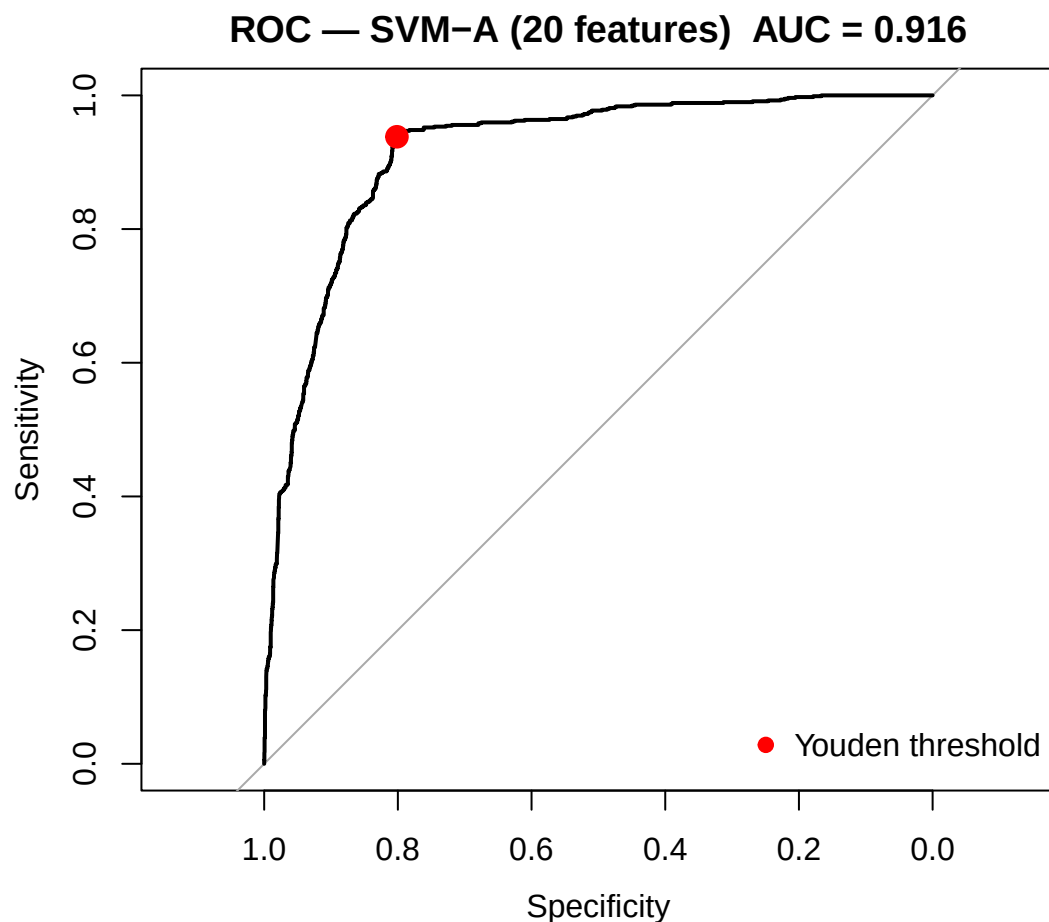
## SVM-A: 20 EDA-selected features

| model              | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   | tp  | fn  | fp  |
|--------------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|-----|-----|-----|
| SVM-A<br>(20 feat) | 0.5            | 0.500     | 0.867    | 0.564       | 0.940       | 0.695     | 0.622 | 0.752        | 0.916 | 446 | 345 | 196 |
| SVM-A<br>(20 feat) | Youden         | 0.127     | 0.828    | 0.938       | 0.801       | 0.533     | 0.680 | 0.870        | 0.916 | 742 | 49  | 649 |

```
plot_cm(task1_y_test, prob_svm_a, 0.5, "SVM-A | threshold 0.5") +
plot_cm(task1_y_test, prob_svm_a, youden_svm_a, "SVM-A | Youden threshold")
```



```
roc_svm_a <- pROC::roc(task1_y_test, prob_svm_a,
                      levels = c("non_high_tc", "high_tc"), quiet = TRUE)
plot(roc_svm_a,
     main = paste0("ROC - SVM-A (20 features) AUC = ", round(auc(roc_svm_a), 3)))
youden_coords_svm_a <- pROC::coords(roc_svm_a, x = "best", best.method = "youden",
                                   ret = c("specificity", "sensitivity"), transpose = FALSE)
points(youden_coords_svm_a$specificity, youden_coords_svm_a$sensitivity,
       pch = 19, col = "red", cex = 1.5)
legend("bottomright", legend = "Youden threshold", col = "red", pch = 19, bty = "n")
```



With 20 features, SVM-A contains several correlated predictors. These duplicate dimensions in the kernel space and can distort the margin, which limits performance compared to the reduced feature set as we will see in SVM-B.

```
preproc_b          <- preProcess(task1_train %>% select(all_of(lr2_features)),
                                method = c("center", "scale"))
train_svm_b_scaled <- predict(preproc_b, task1_train %>% select(all_of(lr2_features)))
test_svm_b_scaled  <- predict(preproc_b, task1_test  %>% select(all_of(lr2_features)))

set.seed(42)
model_svm_b <- svm(
  x      = train_svm_b_scaled,
  y      = task1_y_train,
  kernel = "radial",
  probability = TRUE
)

prob_svm_b <- attr(
  predict(model_svm_b, test_svm_b_scaled, probability = TRUE),
  "probabilities"
)[, "high_tc"]
```

```

svm_b_metrics_05 <- evaluate_clf(task1_y_test, prob_svm_b, threshold = 0.5) %>%
  mutate(threshold_type = "0.5")

youden_svm_b <- get_youden_cutoff(task1_y_test, prob_svm_b)

svm_b_metrics_youden <- evaluate_clf(task1_y_test, prob_svm_b, threshold = youden_svm_b) %>%
  mutate(threshold_type = "Youden")

svm_b_metrics <- bind_rows(svm_b_metrics_05, svm_b_metrics_youden) %>%
  mutate(model = "SVM-B (9 feat)") %>%
  dplyr::select(model, threshold_type, everything())

knitr::kable(svm_b_metrics, digits = 3)

```

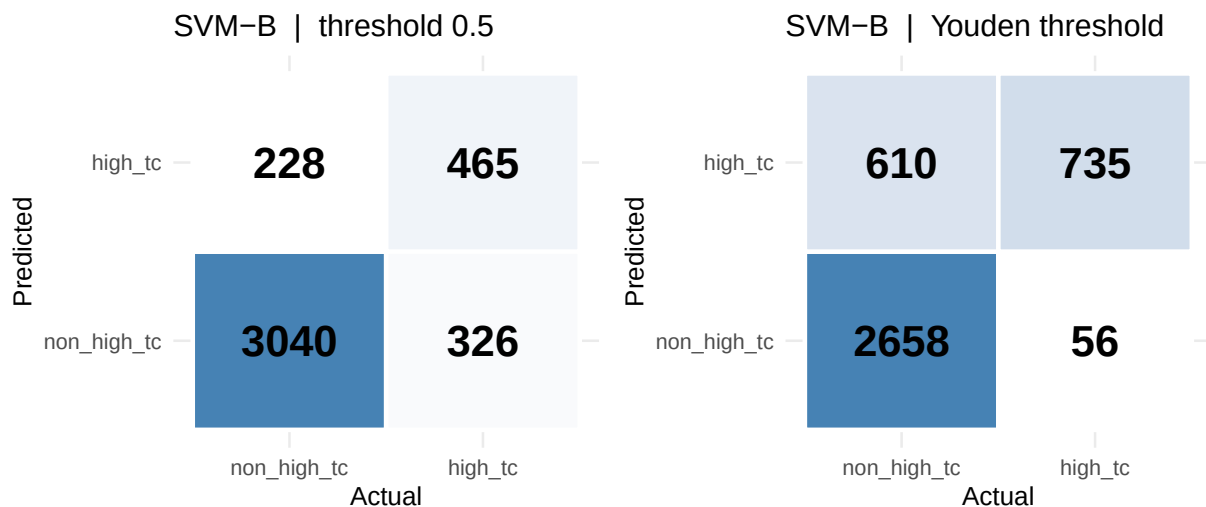
### SVM-B: 9 deduplicated features

| model             | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   | tp  | fn  | fp  |
|-------------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|-----|-----|-----|
| SVM-B<br>(9 feat) | 0.5            | 0.500     | 0.864    | 0.588       | 0.930       | 0.671     | 0.627 | 0.759        | 0.926 | 465 | 326 | 228 |
| SVM-B<br>(9 feat) | Youden         | 0.145     | 0.836    | 0.929       | 0.813       | 0.546     | 0.688 | 0.871        | 0.926 | 735 | 56  | 610 |

```

plot_cm(task1_y_test, prob_svm_b, 0.5, "SVM-B | threshold 0.5") +
plot_cm(task1_y_test, prob_svm_b, youden_svm_b, "SVM-B | Youden threshold")

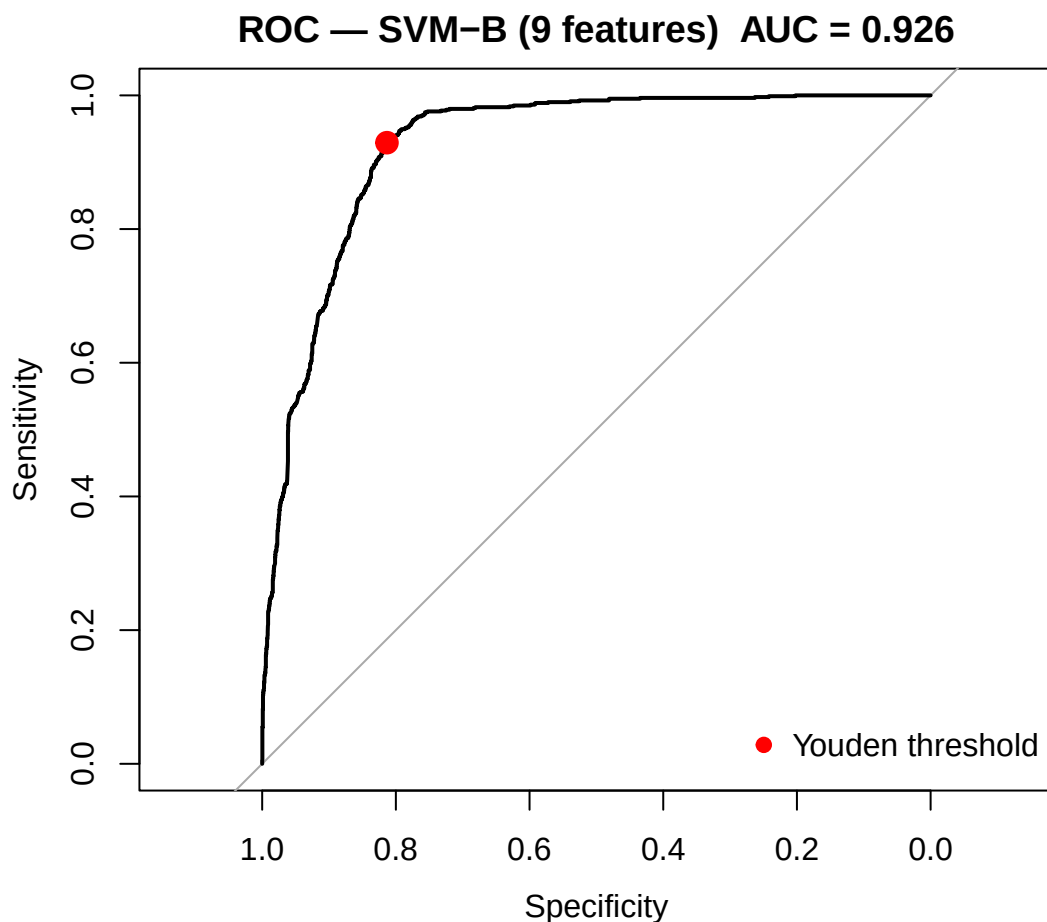
```



```

roc_svm_b <- pROC::roc(task1_y_test, prob_svm_b,
  levels = c("non_high_tc", "high_tc"), quiet = TRUE)
plot(roc_svm_b,
  main = paste0("ROC - SVM-B (9 features) AUC = ", round(auc(roc_svm_b), 3)))
youden_coords_svm_b <- pROC::coords(roc_svm_b, x = "best", best.method = "youden",
  ret = c("specificity", "sensitivity"), transpose = FALSE)
points(youden_coords_svm_b$specificity, youden_coords_svm_b$sensitivity,
  pch = 19, col = "red", cex = 1.5)
legend("bottomright", legend = "Youden threshold", col = "red", pch = 19, bty = "n")

```



With collinearity removed, SVM-B has a cleaner margin in the kernel space. This is the opposite of random forest or decision trees, where more features help — here, removing redundant dimensions improves performance.

```
svm_comparison <- bind_rows(svm_a_metrics, svm_b_metrics) %>%
  dplyr::select(model, threshold_type, threshold, accuracy, sensitivity,
               specificity, precision, f1, balanced_acc, auc)

knitr::kable(svm_comparison, digits = 3)
```

#### Comparison of SVM variants

| model           | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   |
|-----------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|
| SVM-A (20 feat) | 0.5            | 0.500     | 0.867    | 0.564       | 0.940       | 0.695     | 0.622 | 0.752        | 0.916 |
| SVM-A (20 feat) | Youden         | 0.127     | 0.828    | 0.938       | 0.801       | 0.533     | 0.680 | 0.870        | 0.916 |
| SVM-B (9 feat)  | 0.5            | 0.500     | 0.864    | 0.588       | 0.930       | 0.671     | 0.627 | 0.759        | 0.926 |
| SVM-B (9 feat)  | Youden         | 0.145     | 0.836    | 0.929       | 0.813       | 0.546     | 0.688 | 0.871        | 0.926 |

SVM is the least interpretable model evaluated — the RBF kernel maps data into an implicit high-dimensional space and there are no feature coefficients to check. Unlike LR, where collinearity inflates standard errors, in SVM collinearity distorts the margins geometry directly. SVM-B with 9 deduplicated features therefore slightly outperforms SVM-A in AUC (0.926 vs 0.916) and other metrics, confirming that reducing redundancy benefits margin-based classifiers.

## Random Forest

Random forest builds an ensemble method that consists of decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split greedily. This randomisation makes it robust to collinearity — correlated features share the splits across trees rather than distorting a single boundary. The full set of 81 numerical predictors is used, allowing the forest to use weak signals not captured by EDA screening that might prove useful in combination with other features.

Feature importance is measured by mean decrease in Gini impurity (MeanDecreaseGini), which reflects how much each predictor reduces node impurity across all trees.

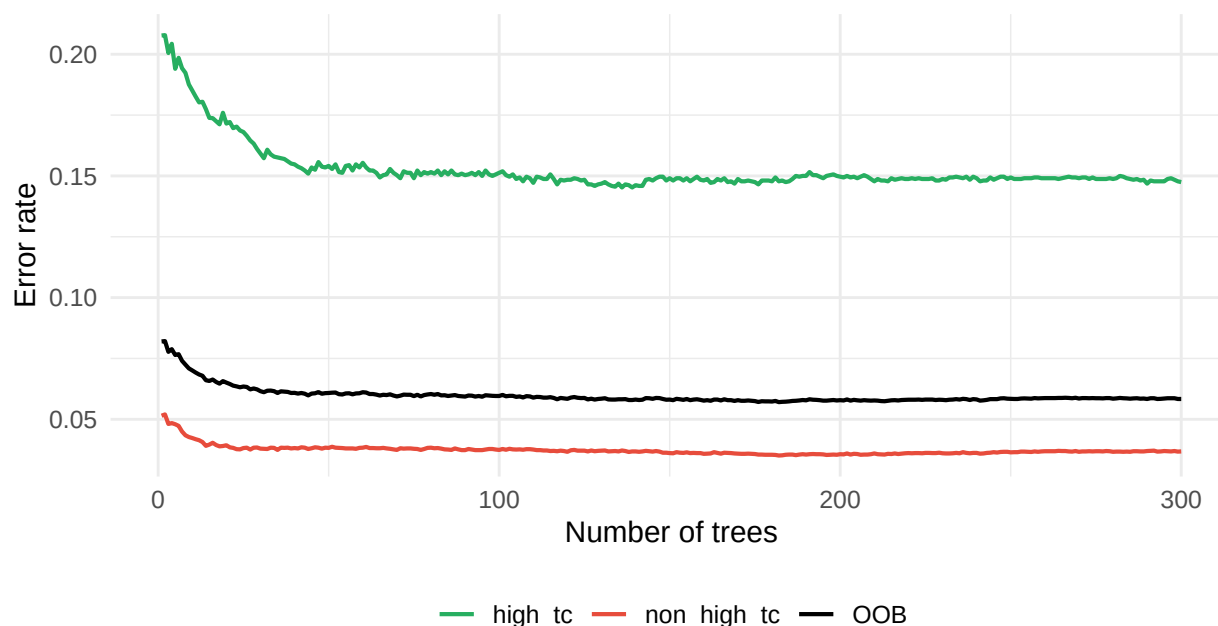
```
set.seed(42)
model_rf <- randomForest(
  tc_class ~ .,
  data      = bind_cols(task1_train %>% select(all_of(numeric_predictors)),
                        tc_class = task1_y_train),
  ntree     = 300,
  mtry      = floor(sqrt(length(numeric_predictors))),
  importance = TRUE
)

cat("RF OOB error:", round(model_rf$err.rate[300, "OOB"], 4), "\n")

## RF OOB error: 0.0583

data.frame(
  trees      = seq_len(nrow(model_rf$err.rate)),
  OOB        = model_rf$err.rate[, "OOB"],
  high_tc    = model_rf$err.rate[, "high_tc"],
  non_high_tc = model_rf$err.rate[, "non_high_tc"]
) %>%
  pivot_longer(-trees, names_to = "type", values_to = "error") %>%
  ggplot(aes(trees, error, color = type)) +
  geom_line(linewidth = 0.8) +
  scale_color_manual(
    values = c("OOB" = "black", "high_tc" = "#27ae60", "non_high_tc" = "#e74c3c"),
    name = NULL
  ) +
  labs(title = "OOB error stabilises as more trees are added",
       x = "Number of trees", y = "Error rate") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
```

## OOB error stabilises as more trees are added



```
prob_rf <- predict(model_rf,
  newdata = task1_test %>% select(all_of(numeric_predictors)),
  type = "prob")[, "high_tc"]

rf_metrics_05 <- evaluate_clf(task1_y_test, prob_rf, threshold = 0.5) %>%
  mutate(threshold_type = "0.5")

youden_rf <- get_youden_cutoff(task1_y_test, prob_rf)

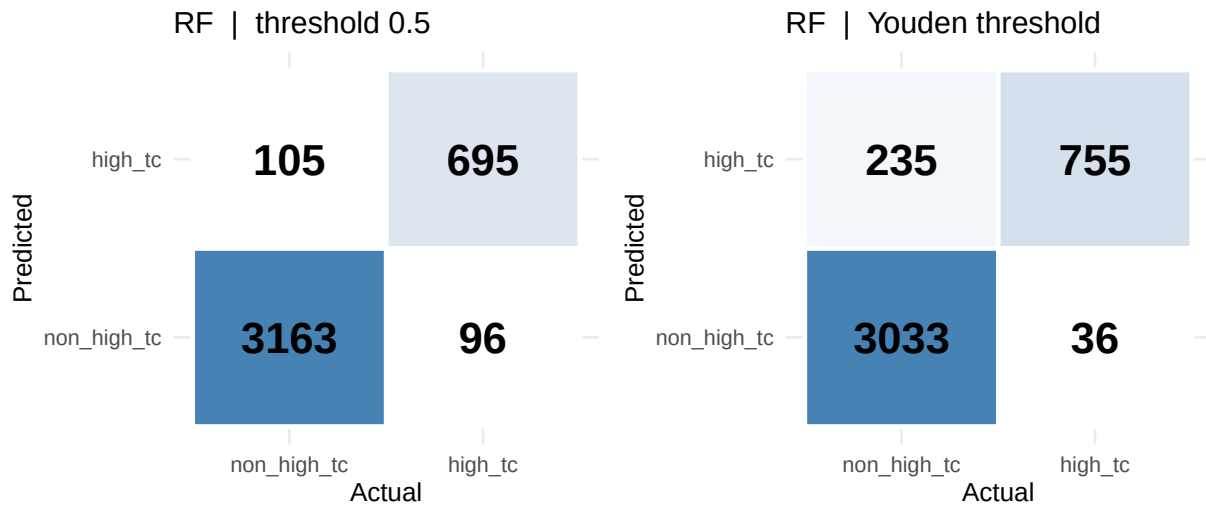
rf_metrics_youden <- evaluate_clf(task1_y_test, prob_rf, threshold = youden_rf) %>%
  mutate(threshold_type = "Youden")

rf_metrics <- bind_rows(rf_metrics_05, rf_metrics_youden) %>%
  mutate(model = "RF (81 feat)") %>%
  dplyr::select(model, threshold_type, everything())

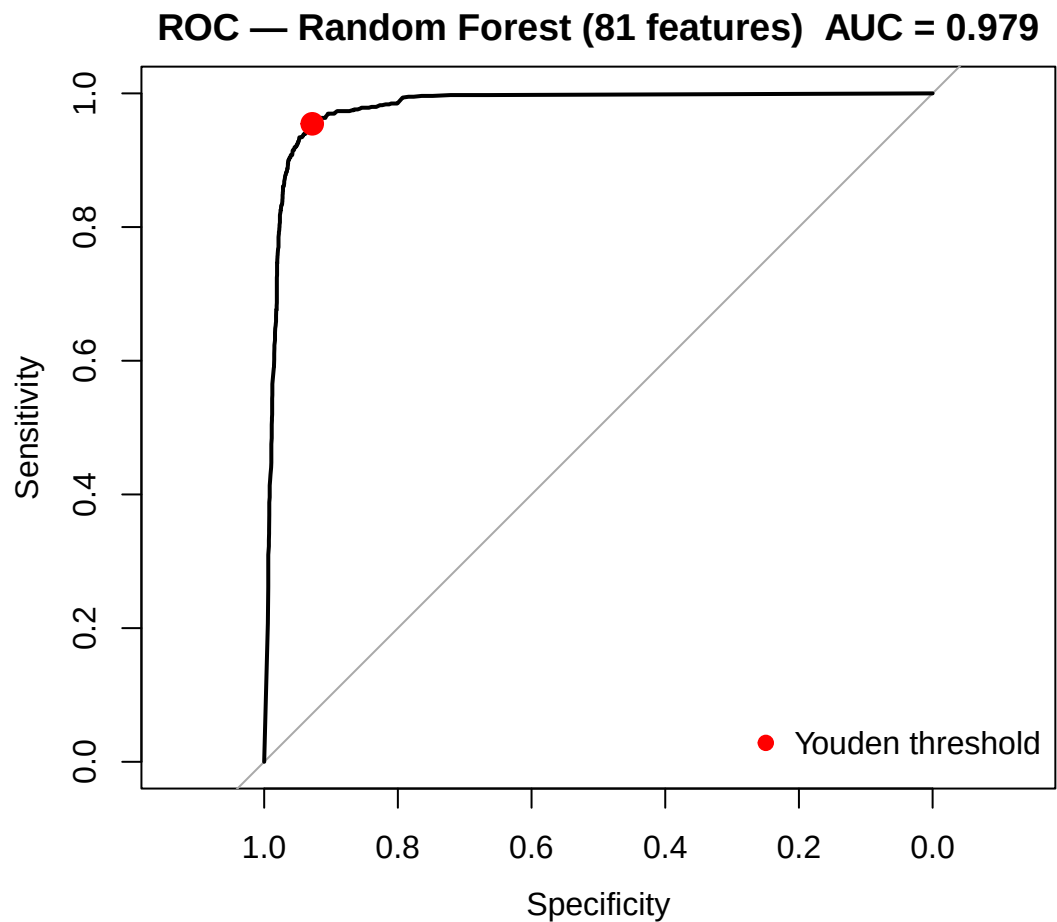
knitr::kable(rf_metrics, digits = 3)
```

| model        | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   | tp  | fn | fp  |
|--------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|-----|----|-----|
| RF (81 feat) | 0.5            | 0.500     | 0.950    | 0.879       | 0.968       | 0.869     | 0.874 | 0.923        | 0.979 | 695 | 96 | 105 |
| RF (81 feat) | Youden         | 0.223     | 0.933    | 0.954       | 0.928       | 0.763     | 0.848 | 0.941        | 0.979 | 755 | 36 | 235 |

```
plot_cm(task1_y_test, prob_rf, 0.5, "RF | threshold 0.5") +
plot_cm(task1_y_test, prob_rf, youden_rf, "RF | Youden threshold")
```

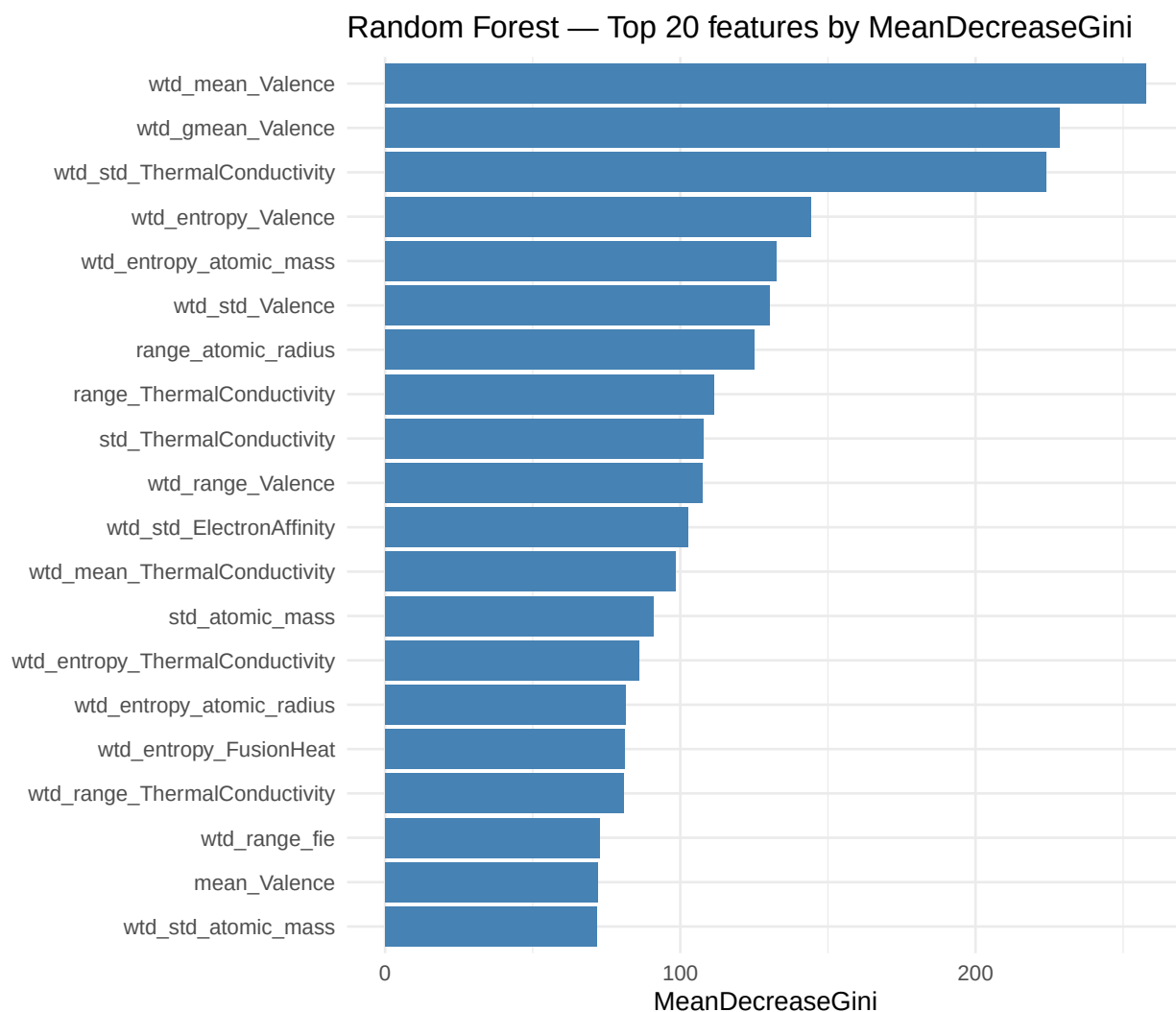


```
roc_rf <- pROC::roc(task1_y_test, prob_rf,
                    levels = c("non_high_tc", "high_tc"), quiet = TRUE)
plot(roc_rf,
     main = paste0("ROC - Random Forest (81 features) AUC = ", round(auc(roc_rf), 3)))
youden_coords_rf <- pROC::coords(roc_rf, x = "best", best.method = "youden",
                                ret = c("specificity", "sensitivity"), transpose = FALSE)
points(youden_coords_rf$specificity, youden_coords_rf$sensitivity,
       pch = 19, col = "red", cex = 1.5)
legend("bottomright", legend = "Youden threshold", col = "red", pch = 19, bty = "n")
```



```
imp_df <- importance(model_rf) %>%
  as.data.frame() %>%
  rownames_to_column("feature") %>%
  arrange(desc(MeanDecreaseGini)) %>%
  slice_head(n = 20)

ggplot(imp_df, aes(x = reorder(feature, MeanDecreaseGini), y = MeanDecreaseGini)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Random Forest - Top 20 features by MeanDecreaseGini",
       x = NULL, y = "MeanDecreaseGini") +
  theme_minimal()
```



Random forest achieved the highest AUC (0.979) of all evaluated models, with the Youden threshold giving sensitivity of 0.954 and specificity of 0.928. With 300 trees and random feature subsampling at each split, the ensemble handles correlated predictors itself — unlike LR and SVM where collinearity is harmful, more features help RF by providing more signal per split.

RF with the Youden threshold is the best overall model. Compared to LR-B, which achieves the highest raw sensitivity (0.970), RF misses only 12 more superconductors (36 vs 24 false negatives), but generates approximately 539 fewer false positives (235 vs 774). In a laboratory screening context this trade-off strongly favours RF — 539 fewer unnecessary experiments is a substantial saving, while 12 additional missed candidates is a smaller cost.

The model is partially interpretable through feature importance, but no directional effect can be read from importance scores alone: a high MeanDecreaseGini only tells us a feature is useful, not whether higher values increase or decrease the probability of `high_tc`.

## Decision Tree

A single decision tree recursively greedily like RF splits the data on the most informative feature at each node. It is the most directly interpretable model in this scenario — the tree diagram shows the exact decision path for any prediction, with explicit thresholds and feature names at every split. The tree is grown with `cp = 0.001` and then pruned back to the complexity parameter that minimises cross-validated error.

```

set.seed(42)
model_dt <- rpart(
  tc_class ~ .,
  data = bind_cols(task1_train %>% select(all_of(numeric_predictors)),
                    tc_class = task1_y_train),
  method = "class",
  control = rpart.control(cp = 0.001)
)

best_cp <- model_dt$cptable[which.min(model_dt$cptable[, "xerror"]), "CP"]
model_dt_pruned <- prune(model_dt, cp = best_cp)
cat("Best cp:", round(best_cp, 6),
    "\n| Terminal nodes:", model_dt_pruned$numresp, "\n")

```

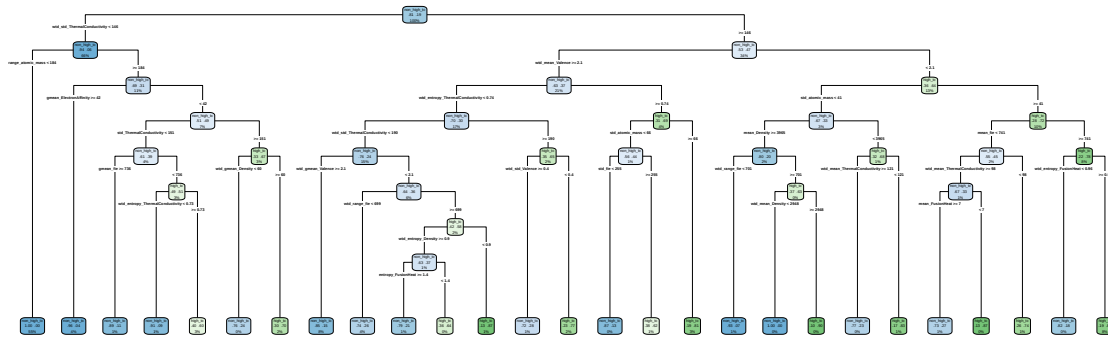
## Best cp: 0.001 | Terminal nodes: 4

```

model_dt_viz <- prune(model_dt, cp = 0.005)
rpart.plot(model_dt_viz, type = 4, extra = 104,
  main = "Decision Tree (simplified view, cp = 0.005)")

```

Decision Tree (simplified view, cp = 0.005)



```

prob_dt <- predict(model_dt_pruned,
  newdata = task1_test %>% select(all_of(numeric_predictors)),
  type = "prob")[, "high_tc"]

dt_metrics_05 <- evaluate_clf(task1_y_test, prob_dt, threshold = 0.5) %>%
  mutate(threshold_type = "0.5")

youden_dt <- get_youden_cutoff(task1_y_test, prob_dt)

```

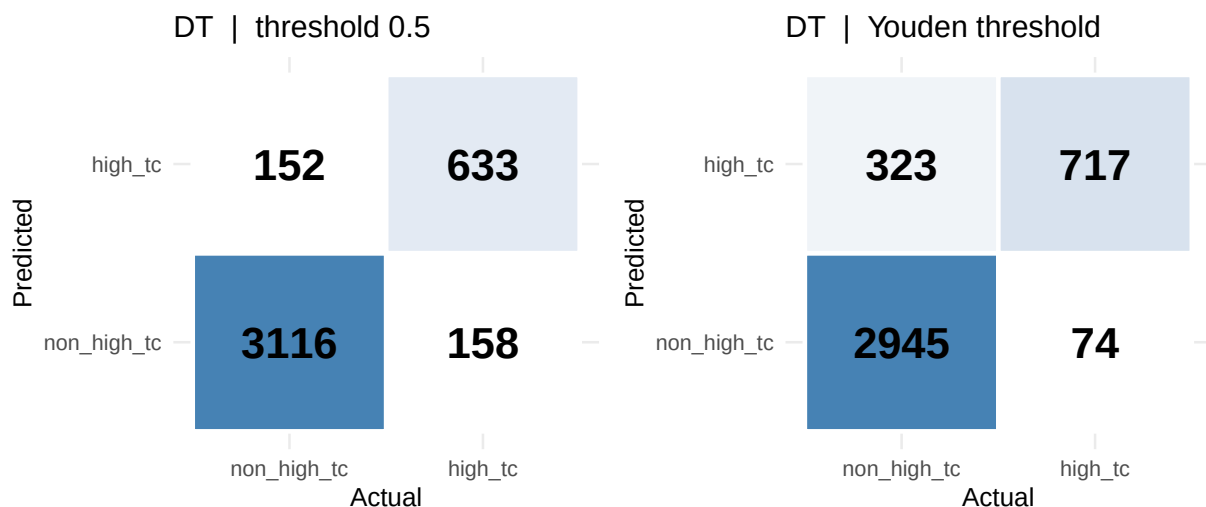
```
dt_metrics_youden <- evaluate_clf(task1_y_test, prob_dt, threshold = youden_dt) %>%
  mutate(threshold_type = "Youden")

dt_metrics <- bind_rows(dt_metrics_05, dt_metrics_youden) %>%
  mutate(model = "DT (81 feat)") %>%
  dplyr::select(model, threshold_type, everything())

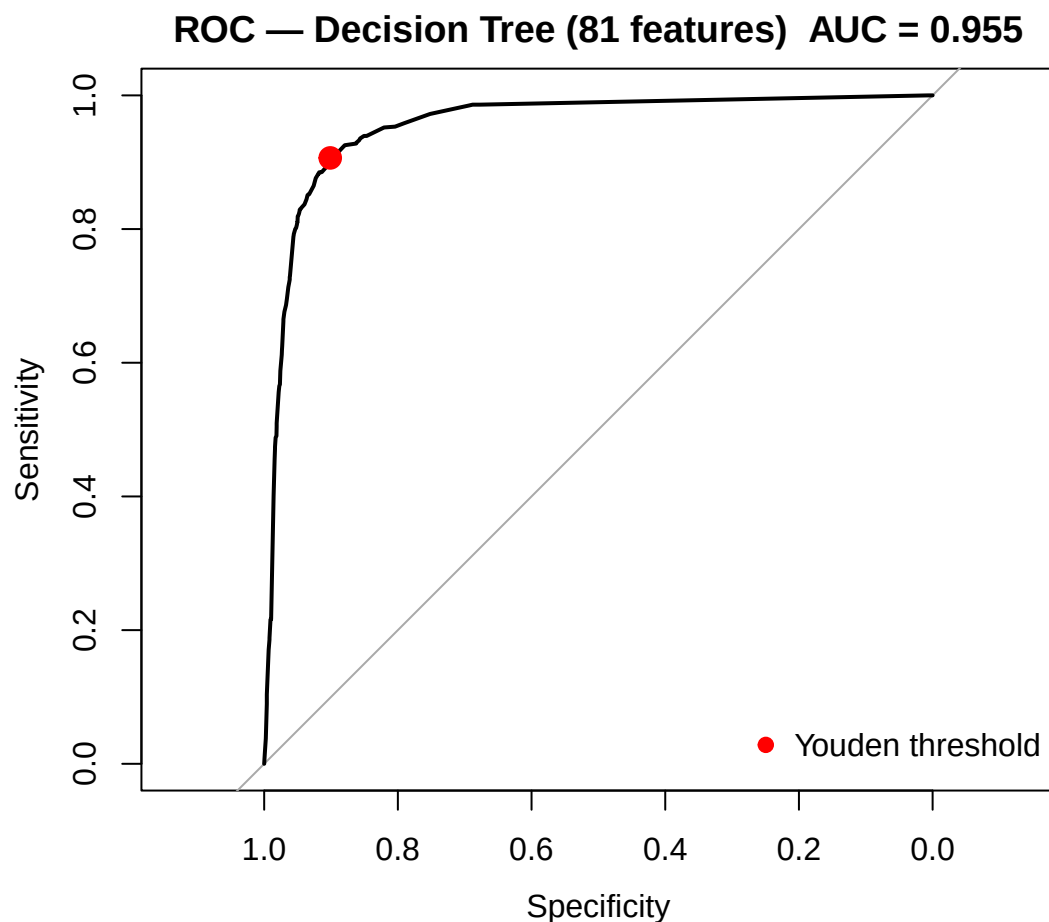
knitr::kable(dt_metrics, digits = 3)
```

| model        | threshold_type | threshold | accuracy | sensitivity | specificity | precision | f1    | balanced_acc | auc   | tp  | fn  | fp  |
|--------------|----------------|-----------|----------|-------------|-------------|-----------|-------|--------------|-------|-----|-----|-----|
| DT (81 feat) | 0.5            | 0.500     | 0.924    | 0.800       | 0.953       | 0.806     | 0.803 | 0.877        | 0.955 | 633 | 158 | 152 |
| DT (81 feat) | Youden         | 0.109     | 0.902    | 0.906       | 0.901       | 0.689     | 0.783 | 0.904        | 0.955 | 717 | 74  | 323 |

```
plot_cm(task1_y_test, prob_dt, 0.5, "DT | threshold 0.5") +
plot_cm(task1_y_test, prob_dt, youden_dt, "DT | Youden threshold")
```



```
roc_dt <- pROC::roc(task1_y_test, prob_dt,
  levels = c("non_high_tc", "high_tc"), quiet = TRUE)
plot(roc_dt,
  main = paste0("ROC - Decision Tree (81 features) AUC = ", round(auc(roc_dt), 3)))
youden_coords_dt <- pROC::coords(roc_dt, x = "best", best.method = "youden",
  ret = c("specificity", "sensitivity"), transpose = FALSE)
points(youden_coords_dt$specificity, youden_coords_dt$sensitivity,
  pch = 19, col = "red", cex = 1.5)
legend("bottomright", legend = "Youden threshold", col = "red", pch = 19, bty = "n")
```



The decision tree achieved an AUC of 0.955, placing it between LR/SVM and RF. It is the second most interpretable model after LR — the tree diagram provides a fully readable decision path. Unlike LR, no p-values are available, but the splits themselves are indication of feature importance. Collinearity is not a concern for DT since each split greedily picks one feature, but a single tree is more prone to overfitting than a forest — the 0.023 AUC gap versus RF reflects that difference perfectly.

### Overall comparison

```
task1_summary <- bind_rows(  
  lr_a_metrics, lr_b_metrics,  
  svm_a_metrics, svm_b_metrics,  
  rf_metrics,  
  dt_metrics  
) %>%  
  dplyr::select(model, threshold_type, accuracy, sensitivity, specificity,  
               f1, balanced_acc, auc, tp, fn, fp)  
  
knitr::kable(task1_summary, digits = 3)
```

| model           | threshold_type | accuracy | sensitivity | specificity | f1    | balanced_acc | auc   | tp  | fn  | fp  |
|-----------------|----------------|----------|-------------|-------------|-------|--------------|-------|-----|-----|-----|
| LR-A (20 feat)  | 0.5            | 0.871    | 0.642       | 0.927       | 0.661 | 0.785        | 0.932 | 508 | 283 | 239 |
| LR-A (20 feat)  | Youden         | 0.828    | 0.946       | 0.799       | 0.682 | 0.872        | 0.932 | 748 | 43  | 656 |
| LR-B (9 feat)   | 0.5            | 0.871    | 0.607       | 0.935       | 0.647 | 0.771        | 0.921 | 480 | 311 | 212 |
| LR-B (9 feat)   | Youden         | 0.804    | 0.970       | 0.763       | 0.658 | 0.867        | 0.921 | 767 | 24  | 773 |
| SVM-A (20 feat) | 0.5            | 0.867    | 0.564       | 0.940       | 0.622 | 0.752        | 0.916 | 446 | 345 | 196 |
| SVM-A (20 feat) | Youden         | 0.828    | 0.938       | 0.801       | 0.680 | 0.870        | 0.916 | 742 | 49  | 649 |
| SVM-B (9 feat)  | 0.5            | 0.864    | 0.588       | 0.930       | 0.627 | 0.759        | 0.926 | 465 | 326 | 228 |
| SVM-B (9 feat)  | Youden         | 0.836    | 0.929       | 0.813       | 0.688 | 0.871        | 0.926 | 735 | 56  | 610 |
| RF (81 feat)    | 0.5            | 0.950    | 0.879       | 0.968       | 0.874 | 0.923        | 0.979 | 695 | 96  | 105 |
| RF (81 feat)    | Youden         | 0.933    | 0.954       | 0.928       | 0.848 | 0.941        | 0.979 | 755 | 36  | 235 |
| DT (81 feat)    | 0.5            | 0.924    | 0.800       | 0.953       | 0.803 | 0.877        | 0.955 | 633 | 158 | 152 |
| DT (81 feat)    | Youden         | 0.902    | 0.906       | 0.901       | 0.783 | 0.904        | 0.955 | 717 | 74  | 323 |

RF with the Youden threshold is the recommended model — it achieves the highest AUC (0.979), best balanced accuracy missing only 36 HTC superconductors, and generates far fewer false positives than LR, making it most practical for laboratory screening. LR-B with the Youden threshold achieves the highest sensitivity (0.970) and fewest missed superconductors (24 false negatives), but this comes at the cost of a very high false positive count (774 vs 235 for RF — 539 more), meaning many unnecessary experiments. For most real-world screening workflows we think this trade-off favours RF, saving huge amounts of time and resources.

Comparing the features selected by EDA (top 20 by SMD) against the features ranked most important by RF (top 20 by MeanDecreaseGini) reveals an interesting split. Ten features appear in both rankings — these are the most robust predictors, capturing signal that is strong regardless of the method used to detect it: `wtd_std_ThermalConductivity`, `range_ThermalConductivity`, `std_ThermalConductivity`, `range_atomic_radius`, `wtd_mean_Valence`, `wtd_gmean_Valence`, `wtd_entropy_atomic_mass`, `wtd_entropy_Valence`, `wtd_entropy_FusionHeat`, and `wtd_entropy_atomic_radius`. The 10 features unique to RF — such as `wtd_std_Valence`, `wtd_mean_ThermalConductivity`, and `wtd_std_ElectronAffinity` — likely gain importance through interactions with other features that only become visible inside the ensemble. The 10 EDA-only features such as `mean_Valence`, `gmean_Density`, and `range_fie` separate the classes well in isolation but contribute less once the forest already used correlated alternatives. The decision tree, being a single greedy model, selects only the little number of features that provide the best and cleanest split — its top splits typically overlap with the core features mentioned, but it ignores the weaker signals that RF leverages across many trees.

In terms of interpretability, the models rank  $LR > DT > RF > SVM$ : logistic regression provides coefficients and p-values, decision trees expose the full decision path, random forest gives only global feature importance without directional effects, and SVM is a black box.

### Scenario 3: Feature Selection Methods

The goal of this scenario is to compare different feature-selection approaches for the same binary classification task. This scenario is motivated by the exploratory data analysis. The EDA showed that several numerical descriptors have strong class-separation ability, but it also revealed substantial redundancy among predictors. In particular, multiple highly ranked features were strongly correlated with each other. This suggests that

some predictors may carry overlapping information, which can reduce model interpretability and make feature selection important.

For this reason, we compare one algorithmic feature-selection method and two embedded regularized methods:

- backward stepwise logistic regression,
- lasso logistic regression,
- elastic net logistic regression.

All methods are applied to the same classification problem and start from the full set of numerical predictors. The aim is not only to compare predictive performance, but also to evaluate how many predictors each method retains and whether a more compact model can be obtained without a substantial loss in performance.

Because the dataset is imbalanced and the `high_tc` class is the minority class, model performance is evaluated using multiple metrics rather than accuracy alone. We report sensitivity, specificity, precision, F1-score and ROC-AUC. In addition to the default threshold of 0.5, we also evaluate the Youden threshold to analyze the trade-off between sensitivity and specificity.

```
# train_df, test_df, numeric_predictors come from the EDA section - no file load needed.
candidate_features <- numeric_predictors

train_task3 <- train_df %>%
  dplyr::select(tc_class, all_of(candidate_features))

test_task3 <- test_df %>%
  dplyr::select(tc_class, all_of(candidate_features))

y_train <- if_else(train_task3$tc_class == "high_tc", 1, 0)
y_test  <- if_else(test_task3$tc_class == "high_tc", 1, 0)

x_train <- model.matrix(tc_class ~ ., data = train_task3)[, -1]
x_test  <- model.matrix(tc_class ~ ., data = test_task3)[, -1]

# Fixed CV folds for reproducible Lasso and Elastic Net results
set.seed(42)

foldid_glmnet <- sample(
  rep(1:10, length.out = nrow(x_train))
)
```

The full set of numerical predictors was used as the initial candidate feature space. This allows the feature-selection methods to operate on the complete high-dimensional representation rather than on a manually pre-filtered subset.

```
# Helper function: evaluate classification performance.
evaluate_classifier <- function(truth, prob, threshold = 0.5) {
  # Convert predicted probabilities to class labels using a selected threshold.
  pred <- if_else(prob >= threshold, "high_tc", "non_high_tc") %>%
    factor(levels = c("non_high_tc", "high_tc"))

  # Ensure that the true labels have the correct factor levels.
  truth <- factor(truth, levels = c("non_high_tc", "high_tc"))

  # Compute confusion matrix metrics.
  cm <- confusionMatrix(pred, truth, positive = "high_tc")

  # Compute ROC curve and AUC.
```

```

roc_obj <- pROC::roc(
  response = truth,
  predictor = prob,
  levels = c("non_high_tc", "high_tc"),
  quiet = TRUE
)

# Return relevant metrics in one tibble.
tibble(
  threshold = threshold,
  accuracy = unname(cm$overall["Accuracy"]),
  sensitivity = unname(cm$byClass["Sensitivity"]),
  specificity = unname(cm$byClass["Specificity"]),
  precision = unname(cm$byClass["Precision"]),
  recall = unname(cm$byClass["Recall"]),
  f1 = unname(cm$byClass["F1"]),
  auc = as.numeric(pROC::auc(roc_obj))
)
}

# Helper function: compute Youden threshold.
get_youden_threshold <- function(truth, prob) {
  # Ensure that the true labels have the correct factor levels.
  truth <- factor(truth, levels = c("non_high_tc", "high_tc"))

  # Build ROC curve.
  roc_obj <- pROC::roc(
    response = truth,
    predictor = prob,
    levels = c("non_high_tc", "high_tc"),
    quiet = TRUE
  )

  # Select the threshold that maximizes Youden's index:
  # sensitivity + specificity - 1.
  pROC::coords(
    roc_obj,
    x = "best",
    best.method = "youden",
    ret = "threshold",
    transpose = FALSE
  ) %>%
  as.numeric() %>%
  first()
}

```

## Backward stepwise logistic regression

As the algorithmic feature-selection method, we used **backward stepwise logistic regression**. This method starts with a full logistic regression model containing all candidate predictors and then removes predictors step by step according to a chosen model-selection criterion. In our case, the criterion was AIC.

Backward selection was chosen because the initial feature space contains 81 numerical predictors, and the goal is to reduce this full predictor set. Starting from the full model is useful in this setting because all candidate

variables are considered at the beginning. In contrast, forward selection starts from an empty model and adds predictors gradually, which may be less suitable when many correlated predictors can provide overlapping information. Mixed stepwise selection, which allows both adding and removing variables, is more flexible, but it is also more complex to interpret. Therefore, backward selection was selected as the representative algorithmic method.

The logistic regression model was fitted using `family = binomial()`, because the target variable is binary (`high_tc` vs. `non_high_tc`). The `stepAIC()` function from the `MASS` package was then used with `direction = "backward"`, meaning that the procedure starts from the full model and iteratively removes predictors if doing so improves the AIC criterion. The argument `trace = FALSE` was used only to suppress the detailed step-by-step output; it does not affect the selected model.

AIC balances model fit and model complexity. Lower AIC indicates a better trade-off between explaining the data and avoiding unnecessary predictors. Therefore, backward stepwise selection provides a model-based way to reduce the predictor set before evaluating the final selected logistic regression model on the test data.

```
full_formula <- as.formula(
  paste("tc_class ~", paste(candidate_features, collapse = " + "))
)

lr_full <- glm(
  full_formula,
  data = train_task3,
  family = binomial()
)

lr_backward <- MASS::stepAIC(
  lr_full,
  direction = "backward",
  trace = FALSE
)

backward_features <- names(coef(lr_backward)) %>%
  setdiff("(Intercept)")

length(backward_features)
```

```
## [1] 66
```

The backward stepwise procedure retained 66 predictors from the original set of 81 numerical predictors. This means that the AIC-based algorithm removed 15 predictors, but the final model still remained relatively large. Therefore, backward selection reduced the feature space only moderately. This suggests that many predictors still contributed to the model according to the AIC criterion, but it also means that the resulting model is not very compact and its interpretability remains limited.

After selecting the predictors, the final backward logistic regression model was evaluated on the test set. The model outputs predicted probabilities for the `high_tc` class. These probabilities were converted into class labels using two decision thresholds.

First, the default threshold of 0.5 was used. This means that a material was classified as `high_tc` only if the predicted probability was at least 50 %. Second, we also evaluated the Youden threshold derived from the ROC curve. This threshold maximizes the expression sensitivity + specificity - 1 and is useful for analyzing the trade-off between correctly detecting `high_tc` materials and correctly rejecting `non_high_tc` materials.

The evaluation includes accuracy, sensitivity, specificity, precision, recall, F1-score and ROC-AUC. Since the dataset is imbalanced and `high_tc` is the minority class, sensitivity, F1-score and AUC are especially important.

```

prob_backward <- predict(
  lr_backward,
  newdata = test_task3,
  type = "response"
)

backward_metrics_05 <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_backward,
  threshold = 0.5
) %>%
  mutate(threshold_type = "0.5")

backward_youden_threshold <- get_youden_threshold(
  truth = test_task3$tc_class,
  prob = prob_backward
)

backward_metrics_youden <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_backward,
  threshold = backward_youden_threshold
) %>%
  mutate(threshold_type = "Youden")

backward_metrics <- bind_rows(
  backward_metrics_05,
  backward_metrics_youden
) %>%
  dplyr::select(
    threshold_type,
    threshold,
    accuracy,
    sensitivity,
    specificity,
    precision,
    recall,
    f1,
    auc
  )

knitr::kable(backward_metrics, digits = 3)

```

| threshold_type | threshold | accuracy | sensitivity | specificity | precision | recall | f1    | auc   |
|----------------|-----------|----------|-------------|-------------|-----------|--------|-------|-------|
| 0.5            | 0.500     | 0.895    | 0.728       | 0.936       | 0.733     | 0.728  | 0.731 | 0.952 |
| Youden         | 0.253     | 0.881    | 0.929       | 0.870       | 0.634     | 0.929  | 0.753 | 0.952 |

With the default threshold of 0.5, the backward logistic regression model achieved an accuracy of 0.895, sensitivity of 0.728, specificity of 0.936, precision of 0.733, F1-score of 0.731 and ROC-AUC of 0.952. The high AUC indicates that the model has strong overall ability to distinguish between **high\_tc** and **non\_high\_tc** samples.

Using the Youden threshold of 0.253 changed the classification trade-off. Sensitivity increased from 0.728 to

0.929, meaning that the model detected a much larger proportion of true `high_tc` samples. However, this improvement came at the cost of lower specificity and precision. Specificity decreased from 0.936 to 0.870, and precision decreased from 0.733 to 0.634. Despite this trade-off, the F1-score increased from 0.731 to 0.753, suggesting that the Youden threshold provides a better balance between precision and recall for this model.

Overall, backward stepwise logistic regression achieved strong predictive performance, but it retained 66 predictors. Therefore, although it performs well, the model remains relatively complex and its interpretability is limited.

### Lasso logistic regression

As the first embedded feature-selection method, we used **lasso logistic regression**. Unlike stepwise selection, lasso performs feature selection directly during model training by adding a penalty to the model coefficients. This penalty can shrink some coefficients exactly to zero, which means that the corresponding predictors are effectively removed from the model.

Lasso was selected because the EDA showed strong redundancy and correlation among several predictors. In such a setting, ordinary logistic regression can become difficult to interpret, while lasso can reduce the predictor space automatically and produce a more compact model. This makes it suitable for comparing predictive performance with model interpretability.

The model was fitted using `cv.glmnet()`, which performs cross-validation to choose the regularization strength. The argument `family = "binomial"` was used because the target variable is binary (`high_tc` vs. `non_high_tc`). The parameter `alpha = 1` specifies pure lasso regularization. The argument `type.measure = "auc"` was used because the classes are imbalanced and AUC evaluates the model's ability to separate the two classes independently of a fixed classification threshold. Finally, `foldid = foldid_glmnet` was used to keep the cross-validation folds consistent across the regularized models, making the lasso and elastic net results more comparable.

```
cv_lasso <- cv.glmnet(  
  x = x_train,  
  y = y_train,  
  family = "binomial",  
  alpha = 1,  
  type.measure = "auc",  
  foldid = foldid_glmnet  
)
```

After fitting the cross-validated lasso model, we first evaluated the model using `lambda.min`. This value of `lambda` corresponds to the regularization strength that achieved the best cross-validated AUC. Therefore, it is expected to prioritize predictive performance rather than model simplicity.

The selected lasso model was then used to obtain predicted probabilities on the test set. As with the previous model, performance was evaluated using both the default threshold of 0.5 and the Youden threshold derived from the ROC curve.

```
lasso_coef_min <- coef(cv_lasso, s = "lambda.min")  
  
lasso_features_min <- rownames(lasso_coef_min)[as.vector(lasso_coef_min != 0)] %>%  
  setdiff("(Intercept)")  
  
prob_lasso_min <- predict(  
  cv_lasso,  
  newx = x_test,  
  s = "lambda.min",  
  type = "response"  
) %>%
```

```

as.vector()

lasso_metrics_min_05 <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_lasso_min,
  threshold = 0.5
)

lasso_youden_threshold_min <- get_youden_threshold(
  truth = test_task3$tc_class,
  prob = prob_lasso_min
)

lasso_metrics_min_youden <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_lasso_min,
  threshold = lasso_youden_threshold_min
)

lasso_metrics_min <- bind_rows(
  lasso_metrics_min_05 %>%
    mutate(lambda_type = "lambda.min", threshold_type = "0.5"),
  lasso_metrics_min_youden %>%
    mutate(lambda_type = "lambda.min", threshold_type = "Youden")
) %>%
  dplyr::select(lambda_type, threshold_type, everything())

length(lasso_features_min)

## [1] 73

knitr::kable(lasso_metrics_min, digits = 3)

```

| lambda_type | threshold_type | threshold | accuracy | sensitivity | specificity | precision | recall | f1    | auc  |
|-------------|----------------|-----------|----------|-------------|-------------|-----------|--------|-------|------|
| lambda.min  | 0.5            | 0.500     | 0.891    | 0.698       | 0.937       | 0.729     | 0.698  | 0.713 | 0.95 |
| lambda.min  | Youden         | 0.284     | 0.885    | 0.915       | 0.878       | 0.644     | 0.915  | 0.756 | 0.95 |

Using `lambda.min`, the lasso model retained 73 predictors. This indicates that the selected penalty was relatively weak and did not produce a very sparse model. At the default threshold of 0.5, the model achieved an accuracy of 0.891, sensitivity of 0.698, specificity of 0.937, F1-score of 0.713 and ROC-AUC of 0.950.

When the Youden threshold was applied, the decision threshold decreased to 0.284. This substantially increased sensitivity from 0.698 to 0.915, meaning that the model detected a much larger proportion of true `high_tc` samples. However, this came at the cost of lower precision and specificity. Precision decreased from 0.729 to 0.644 and specificity decreased from 0.937 to 0.878. The F1-score improved from 0.713 to 0.756, suggesting that the Youden threshold provides a better balance between precision and recall for this lasso model.

Although the predictive performance is strong, the model retained 73 predictors. Therefore, `lambda.min` does not provide a sufficiently compact model for interpretability. For this reason, we also evaluated `lambda.1se`, which applies a stronger regularization penalty. It represents the simplest model whose cross-validated performance is still within one standard error of the best model. Therefore, `lambda.1se` is useful for studying the trade-off between predictive performance and model interpretability.

Compared with `lambda.min`, this setting is expected to retain fewer predictors, possibly at the cost of slightly lower predictive performance.

```
lasso_coef_1se <- coef(cv_lasso, s = "lambda.1se")

lasso_features_1se <- rownames(lasso_coef_1se)[as.vector(lasso_coef_1se != 0)] %>%
  setdiff("(Intercept)")

prob_lasso_1se <- predict(
  cv_lasso,
  newx = x_test,
  s = "lambda.1se",
  type = "response"
) %>%
  as.vector()

lasso_metrics_1se_05 <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_lasso_1se,
  threshold = 0.5
)

lasso_youden_threshold_1se <- get_youden_threshold(
  truth = test_task3$tc_class,
  prob = prob_lasso_1se
)

lasso_metrics_1se_youden <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_lasso_1se,
  threshold = lasso_youden_threshold_1se
)

lasso_metrics_1se <- bind_rows(
  lasso_metrics_1se_05 %>%
    mutate(lambda_type = "lambda.1se", threshold_type = "0.5"),
  lasso_metrics_1se_youden %>%
    mutate(lambda_type = "lambda.1se", threshold_type = "Youden")
) %>%
  dplyr::select(lambda_type, threshold_type, everything())

length(lasso_features_1se)

## [1] 63

knitr::kable(lasso_metrics_1se, digits = 3)
```

| lambda_type | threshold_type | threshold | accuracy | sensitivity | specificity | precision | recall | f1    | auc   |
|-------------|----------------|-----------|----------|-------------|-------------|-----------|--------|-------|-------|
| lambda.1se  | 0.5            | 0.500     | 0.886    | 0.679       | 0.936       | 0.72      | 0.679  | 0.699 | 0.947 |
| lambda.1se  | Youden         | 0.296     | 0.879    | 0.918       | 0.870       | 0.63      | 0.918  | 0.747 | 0.947 |

Using `lambda.1se`, the lasso model retained 63 predictors, which is fewer than the 73 predictors retained by `lambda.min`. This confirms that the stronger regularization reduced model complexity. However, 63 predictors is still a large number, so the model cannot be considered highly sparse or easily interpretable.

At the default threshold of 0.5, the `lambda.1se` model achieved an accuracy of 0.886, sensitivity of 0.679, specificity of 0.936, F1-score of 0.699 and ROC-AUC of 0.947. Compared with `lambda.min`, the model became slightly simpler, but its predictive performance also decreased slightly.

Using the Youden threshold of 0.296 increased sensitivity from 0.679 to 0.918. This again shows that lowering the threshold improves detection of the minority `high_tc` class. However, precision decreased to 0.630 and specificity decreased to 0.870. The F1-score increased to 0.747, which is close to the `lambda.min` Youden result, but with fewer retained predictors.

Overall, lasso with `lambda.1se` provides a better interpretability-performance trade-off than `lambda.min`, but the reduction is still limited. Even with stronger regularization, the model retained many predictors, suggesting that the predictive signal is spread across a larger number of material descriptors rather than concentrated in only a few features.

```
lasso_summary <- bind_rows(
  lasso_metrics_min %>%
    mutate(
      lambda_type = "lambda.min",
      retained_features = length(lasso_features_min)
    ),
  lasso_metrics_1se %>%
    mutate(
      lambda_type = "lambda.1se",
      retained_features = length(lasso_features_1se)
    )
) %>%
  dplyr::select(
    lambda_type,
    threshold_type,
    retained_features,
    threshold,
    accuracy,
    sensitivity,
    specificity,
    precision,
    recall,
    f1,
    auc
  )

knitr::kable(lasso_summary, digits = 3)
```

| lambda_type | threshold_type | retained_features | threshold | accuracy | sensitivity | specificity | precision | recall | f1    | auc   |
|-------------|----------------|-------------------|-----------|----------|-------------|-------------|-----------|--------|-------|-------|
| lambda.min  | 0.5            | 73                | 0.500     | 0.891    | 0.698       | 0.937       | 0.729     | 0.698  | 0.713 | 0.950 |
| lambda.min  | Youden         | 73                | 0.284     | 0.885    | 0.915       | 0.878       | 0.644     | 0.915  | 0.756 | 0.950 |
| lambda.1se  | 0.5            | 63                | 0.500     | 0.886    | 0.679       | 0.936       | 0.720     | 0.679  | 0.699 | 0.947 |
| lambda.1se  | Youden         | 63                | 0.296     | 0.879    | 0.918       | 0.870       | 0.630     | 0.918  | 0.747 | 0.947 |

The summary confirms that `lambda.min` achieves slightly stronger predictive performance, but retains more predictors. The `lambda.1se` setting reduces the number of predictors and keeps a similar AUC, but the model is still relatively large. In both cases, the Youden threshold improves sensitivity and F1-score, which is useful for detecting the minority `high_tc` class.

## Elastic net logistic regression

As the second embedded feature-selection method, we used **elastic net logistic regression**. Elastic net combines the lasso and ridge penalties. This is useful when predictors are strongly correlated, because pure lasso may select only one variable from a correlated group, while elastic net can keep groups of related predictors together.

This method was selected because the EDA showed strong correlations between several material descriptors. Therefore, elastic net is a suitable alternative to lasso: it still performs regularization and feature selection, but it can be more stable in the presence of correlated predictors.

The model was fitted using `cv.glmnet()`. We used `family = "binomial"` because the target variable is binary. The parameter `alpha` controls the balance between ridge and lasso regularization. Values closer to 0 behave more like ridge, while values closer to 1 behave more like lasso. We tested three values: 0.2, 0.5, and 0.8. The value with the highest cross-validated AUC was selected. As in the lasso model, `type.measure = "auc"` was used because the dataset is imbalanced, and `foldid = foldid_glmnet` ensured that the same cross-validation folds were used for a fair comparison.

```
alpha_grid <- c(0.2, 0.5, 0.8)

enet_cv_results <- tibble(alpha = alpha_grid) %>%
  mutate(
    cv_fit = purrr::map(
      alpha,
      ~ cv.glmnet(
        x = x_train,
        y = y_train,
        family = "binomial",
        alpha = .x,
        type.measure = "auc",
        foldid = foldid_glmnet
      )
    ),
    best_auc = purrr::map_dbl(cv_fit, ~ max(.x$cvm))
  ) %>%
  arrange(desc(best_auc))

enet_cv_results
```

```
## # A tibble: 3 x 3
##   alpha cv_fit      best_auc
##   <dbl> <list>         <dbl>
## 1   0.8 <cv.glmnt>      0.949
## 2   0.5 <cv.glmnt>      0.948
## 3   0.2 <cv.glmnt>      0.946

best_enet <- enet_cv_results$cv_fit[[1]]
best_alpha <- enet_cv_results$alpha[[1]]

best_alpha

## [1] 0.8
```

Among the tested alpha values, `alpha = 0.8` achieved the highest cross-validated AUC. This means that the best elastic net model was closer to lasso than to ridge, but still included a ridge component that can help with correlated predictors.

Similarly to lasso, the elastic net model was first evaluated using `lambda.min`. This value corresponds to

the regularization strength with the best cross-validated AUC. Therefore, it mainly prioritizes predictive performance rather than model simplicity.

The selected model was evaluated on the test set using both the default threshold of 0.5 and the Youden threshold.

```
enet_coef_min <- coef(best_enet, s = "lambda.min")

enet_features_min <- rownames(enet_coef_min)[as.vector(enet_coef_min != 0)] %>%
  setdiff("(Intercept)")

prob_enet_min <- predict(
  best_enet,
  newx = x_test,
  s = "lambda.min",
  type = "response"
) %>%
  as.vector()

enet_metrics_min_05 <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_enet_min,
  threshold = 0.5
) %>%
  mutate(
    lambda_type = "lambda.min",
    threshold_type = "0.5",
    retained_features = length(enet_features_min)
  )

enet_youden_threshold_min <- get_youden_threshold(
  truth = test_task3$tc_class,
  prob = prob_enet_min
)

enet_metrics_min_youden <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_enet_min,
  threshold = enet_youden_threshold_min
) %>%
  mutate(
    lambda_type = "lambda.min",
    threshold_type = "Youden",
    retained_features = length(enet_features_min)
  )

length(enet_features_min)
```

```
## [1] 76
```

Using `lambda.min`, the elastic net model retained 76 predictors. This means that the model was not sparse, even though regularization was applied. At the default threshold of 0.5, it achieved an accuracy of 0.890, sensitivity of 0.694, specificity of 0.938, F1-score of 0.711 and ROC-AUC of 0.950.

After applying the Youden threshold of 0.292, sensitivity increased from 0.694 to 0.914. This improved the detection of the minority `high_tc` class, but precision decreased from 0.729 to 0.646 and specificity decreased

from 0.938 to 0.879. The F1-score increased from 0.711 to 0.757. Therefore, the Youden threshold again provided a better balance between precision and recall, although the model still retained many predictors.

Next, we evaluated the elastic net model with `lambda.1se`. This setting applies a stronger regularization penalty than `lambda.min` and is expected to produce a simpler model. The goal is to check whether elastic net can reduce the number of retained predictors while preserving most of the predictive performance.

```
enet_coef_1se <- coef(best_enet, s = "lambda.1se")

enet_features_1se <- rownames(enet_coef_1se)[as.vector(enet_coef_1se != 0)] %>%
  setdiff("(Intercept)")

prob_enet_1se <- predict(
  best_enet,
  newx = x_test,
  s = "lambda.1se",
  type = "response"
) %>%
  as.vector()

enet_metrics_1se_05 <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_enet_1se,
  threshold = 0.5
) %>%
  mutate(
    lambda_type = "lambda.1se",
    threshold_type = "0.5",
    retained_features = length(enet_features_1se)
  )

enet_youden_threshold_1se <- get_youden_threshold(
  truth = test_task3$tc_class,
  prob = prob_enet_1se
)

enet_metrics_1se_youden <- evaluate_classifier(
  truth = test_task3$tc_class,
  prob = prob_enet_1se,
  threshold = enet_youden_threshold_1se
) %>%
  mutate(
    lambda_type = "lambda.1se",
    threshold_type = "Youden",
    retained_features = length(enet_features_1se)
  )

length(enet_features_1se)
```

```
## [1] 67
```

Using `lambda.1se`, the elastic net model retained 67 predictors, which is fewer than the 76 predictors retained by `lambda.min`. However, this is still a large number of predictors, so the model remains relatively complex.

At the default threshold of 0.5, the model achieved an accuracy of 0.886, sensitivity of 0.679, specificity of 0.937, F1-score of 0.700 and ROC-AUC of 0.948. Compared with `lambda.min`, the model became slightly

simpler, but the predictive performance decreased only slightly.

Using the Youden threshold of 0.306 increased sensitivity to 0.913 and improved the F1-score to 0.751. This performance is close to the `lambda.min` version, but with fewer retained predictors. Therefore, `lambda.1se` provides a slightly better interpretability-performance trade-off than `lambda.min`, although the feature reduction is still moderate.

```
enet_summary <- bind_rows(
  enet_metrics_min_05,
  enet_metrics_min_youden,
  enet_metrics_1se_05,
  enet_metrics_1se_youden
) %>%
  mutate(
    method = paste0("Elastic Net LR (alpha=", best_alpha, ")")
  ) %>%
  dplyr::select(
    method,
    lambda_type,
    threshold_type,
    retained_features,
    threshold,
    accuracy,
    sensitivity,
    specificity,
    precision,
    recall,
    f1,
    auc
  )

knitr::kable(enet_summary, digits = 3)
```

| method                        | lambda_type | threshold_type | retained_features | threshold | accuracy | sensitivity | specificity | precision | recall | f1    | auc   |
|-------------------------------|-------------|----------------|-------------------|-----------|----------|-------------|-------------|-----------|--------|-------|-------|
| Elastic Net LR<br>(alpha=0.8) | lambda.min  | 0.5            | 76                | 0.500     | 0.890    | 0.694       | 0.938       | 0.729     | 0.694  | 0.711 | 0.950 |
| Elastic Net LR<br>(alpha=0.8) | lambda.min  | Youden         | 76                | 0.292     | 0.886    | 0.914       | 0.879       | 0.646     | 0.914  | 0.757 | 0.950 |
| Elastic Net LR<br>(alpha=0.8) | lambda.1se  | 0.5            | 67                | 0.500     | 0.886    | 0.679       | 0.937       | 0.722     | 0.679  | 0.700 | 0.948 |
| Elastic Net LR<br>(alpha=0.8) | lambda.1se  | Youden         | 67                | 0.306     | 0.882    | 0.913       | 0.875       | 0.638     | 0.913  | 0.751 | 0.948 |

The elastic net summary shows a similar pattern to lasso. The `lambda.min` setting achieved slightly stronger performance but retained more predictors, while `lambda.1se` reduced the number of predictors with only a small decrease in AUC and F1-score. However, even the more regularized elastic net model retained 67 predictors, which suggests that elastic net did not produce a highly sparse model for this dataset.

Compared with lasso, elastic net retained more predictors. This is expected, because elastic net tends to keep

groups of correlated variables together instead of selecting only one variable from such a group.

### Overall comparison of feature-selection methods

```
task3_summary <- bind_rows(
  backward_metrics %>%
    mutate(
      method = "Backward LR",
      lambda_type = "AIC",
      retained_features = length(backward_features)
    ) %>%
  dplyr::select(
    method,
    lambda_type,
    threshold_type,
    retained_features,
    threshold,
    accuracy,
    sensitivity,
    specificity,
    precision,
    recall,
    f1,
    auc
  ),
  lasso_summary %>%
    mutate(
      method = "Lasso LR"
    ) %>%
  dplyr::select(
    method,
    lambda_type,
    threshold_type,
    retained_features,
    threshold,
    accuracy,
    sensitivity,
    specificity,
    precision,
    recall,
    f1,
    auc
  ),
  enet_summary
)

knitr::kable(task3_summary, digits = 3)
```

| method      | lambda_type | threshold_type | retained_features | threshold | accuracy | sensitivity | specificity | precision | recall | f1    | auc   |
|-------------|-------------|----------------|-------------------|-----------|----------|-------------|-------------|-----------|--------|-------|-------|
| Backward LR | AIC         | 0.5            | 66                | 0.500     | 0.895    | 0.728       | 0.936       | 0.733     | 0.728  | 0.731 | 0.952 |
| Backward LR | AIC         | Youden         | 66                | 0.253     | 0.881    | 0.929       | 0.870       | 0.634     | 0.929  | 0.753 | 0.952 |
| Lasso LR    | lambda.1    | 0.5            | 73                | 0.500     | 0.891    | 0.698       | 0.937       | 0.729     | 0.698  | 0.713 | 0.950 |
| Lasso LR    | lambda.1    | Youden         | 73                | 0.284     | 0.885    | 0.915       | 0.878       | 0.644     | 0.915  | 0.756 | 0.950 |

| method                        | lambda_type | threshold_type | retained_features | threshold | accuracy | sensitivity | specificity | precision | recall | f1    | auc   |
|-------------------------------|-------------|----------------|-------------------|-----------|----------|-------------|-------------|-----------|--------|-------|-------|
| Lasso LR                      | lambda.1se  | 0.5            | 63                | 0.500     | 0.886    | 0.679       | 0.936       | 0.720     | 0.679  | 0.699 | 0.947 |
| Lasso LR                      | lambda.1se  | Youden         | 63                | 0.296     | 0.879    | 0.918       | 0.870       | 0.630     | 0.918  | 0.747 | 0.947 |
| Elastic Net LR                | lambda.min  | 0.5            | 76                | 0.500     | 0.890    | 0.694       | 0.938       | 0.729     | 0.694  | 0.711 | 0.950 |
| (alpha=0.8)<br>Elastic Net LR | lambda.min  | Youden         | 76                | 0.292     | 0.886    | 0.914       | 0.879       | 0.646     | 0.914  | 0.757 | 0.950 |
| (alpha=0.8)<br>Elastic Net LR | lambda.1se  | 0.5            | 67                | 0.500     | 0.886    | 0.679       | 0.937       | 0.722     | 0.679  | 0.700 | 0.948 |
| (alpha=0.8)<br>Elastic Net LR | lambda.1se  | Youden         | 67                | 0.306     | 0.882    | 0.913       | 0.875       | 0.638     | 0.913  | 0.751 | 0.948 |

The final comparison shows that all feature-selection methods achieved strong predictive performance, with ROC-AUC values close to or above 0.95 for the less regularized variants. However, the reduction in the number of predictors was only moderate.

Backward stepwise logistic regression retained 66 predictors and achieved the highest AUC among the feature-selection models. With the Youden threshold, it reached sensitivity of 0.929 and F1-score of 0.753. Lasso with `lambda.min` retained more predictors, while lasso with `lambda.1se` reduced the number of retained predictors and preserved similar performance. Elastic net showed a similar pattern, but retained more predictors than lasso, which is expected because elastic net tends to keep correlated groups of variables together.

Across the evaluated methods, the Youden threshold consistently increased sensitivity to the minority `high_tc` class and improved F1-score, but at the cost of lower precision and specificity. This means that the Youden threshold is more suitable when the goal is to detect as many potentially high-Tc materials as possible, while the default threshold of 0.5 is more conservative.

Overall, the best performance-oriented feature-selection model was backward stepwise logistic regression, while lasso with `lambda.1se` provided a better compromise between model compactness and predictive performance. Nevertheless, none of the methods produced a very sparse model, suggesting that the predictive signal is distributed across many material descriptors.

### Feature retention comparison

Besides comparing predictive performance, we also compared which predictors were retained by the feature-selection methods. This is important because several models retained a relatively large number of predictors, and the overlap between methods can indicate which features are more stable.

For each predictor, we recorded whether it was selected by backward stepwise logistic regression, lasso with `lambda.min`, lasso with `lambda.1se`, elastic net with `lambda.min`, and elastic net with `lambda.1se`. Instead of displaying the full table of all 81 predictors, we summarize the number of retained predictors per method and list the predictors selected by all evaluated feature-selection variants. The full feature-retention table was exported as `task3_feature_retention.csv`.

```
feature_retention <- tibble(feature = candidate_features) %>%
  mutate(
    backward = feature %in% backward_features,
    lasso_min = feature %in% lasso_features_min,
    lasso_1se = feature %in% lasso_features_1se,
    elastic_net_min = feature %in% enet_features_min,
    elastic_net_1se = feature %in% enet_features_1se,
```

```

    selected_count = backward + lasso_min + lasso_1se + elastic_net_min + elastic_net_1se
  ) %>%
  arrange(desc(selected_count), feature)

write_csv(feature_retention, "task3_feature_retention.csv")

stable_features <- feature_retention %>%
  filter(selected_count == 5) %>%
  arrange(desc(selected_count), feature)

knitr::kable(stable_features, digits = 3)

```

| feature                         | backward | lasso_min | lasso_1se | elastic_net_min | elastic_net_1se | selected_count |
|---------------------------------|----------|-----------|-----------|-----------------|-----------------|----------------|
| entropy_Density                 | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| entropy_ThermalConductivity     | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| entropy_Valence                 | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| entropy_atomic_mass             | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| gmean_Density                   | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| gmean_ThermalConductivity       | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| gmean_Valence                   | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| gmean_atomic_radius             | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| mean_Density                    | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| mean_FusionHeat                 | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| mean_atomic_mass                | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| mean_atomic_radius              | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| mean_fie                        | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| number_of_elements              | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| range_ElectronAffinity          | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| range_atomic_mass               | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| range_atomic_radius             | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| range_fie                       | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| std_Density                     | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| std_ElectronAffinity            | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| std_Valence                     | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| std_atomic_radius               | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| std_fie                         | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_entropy_Density             | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_entropy_FusionHeat          | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_entropy_ThermalConductivity | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_entropy_Valence             | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_entropy_atomic_mass         | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_entropy_atomic_radius       | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_entropy_fie                 | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_gmean_ElectronAffinity      | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_gmean_ThermalConductivity   | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_gmean_Valence               | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_gmean_atomic_mass           | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_gmean_fie                   | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_mean_ElectronAffinity       | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_mean_FusionHeat             | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_mean_atomic_mass            | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_mean_atomic_radius          | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_range_Density               | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |

| feature                       | backward | lasso_min | lasso_1se | elastic_net_min | elastic_net_1se | selected_count |
|-------------------------------|----------|-----------|-----------|-----------------|-----------------|----------------|
| wtd_range_ThermalConductivity | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_range_Valence             | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_range_fie                 | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_std_Density               | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_std_ElectronAffinity      | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_std_ThermalConductivity   | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_std_Valence               | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_std_atomic_mass           | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_std_atomic_radius         | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |
| wtd_std_fie                   | TRUE     | TRUE      | TRUE      | TRUE            | TRUE            | 5              |

```
feature_selection_counts <- feature_retention %>%
  summarise(
    backward = sum(backward),
    lasso_min = sum(lasso_min),
    lasso_1se = sum(lasso_1se),
    elastic_net_min = sum(elastic_net_min),
    elastic_net_1se = sum(elastic_net_1se)
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = "method",
    values_to = "retained_features"
  ) %>%
  arrange(retained_features)
```

```
feature_selection_counts
```

```
## # A tibble: 5 x 2
##   method      retained_features
##   <chr>          <int>
## 1 lasso_1se         63
## 2 backward         66
## 3 elastic_net_1se  67
## 4 lasso_min        73
## 5 elastic_net_min  76
```

The number of retained predictors confirms that none of the methods produced a very sparse model. Lasso with `lambda.1se` retained the fewest predictors among the standard evaluated methods, while elastic net retained more predictors, especially under `lambda.min`. This supports the conclusion that the predictive information is spread across many descriptors rather than concentrated in only a small subset of variables.

## Final Model Selection

The final model was selected based on predictive performance, class-imbalance handling, and practical usability. Since the `high_tc` class is the minority class, accuracy alone was not used as the main decision criterion. Instead, we focused mainly on ROC-AUC, F1-score, balanced accuracy, sensitivity and the trade-off between sensitivity and specificity.

Across Scenario 1, Random Forest achieved the strongest overall performance. It reached the highest ROC-AUC and F1-score and provided the best balance between correctly detecting `high_tc` materials and avoiding excessive false positives. With the Youden threshold, Random Forest achieved high sensitivity while

still maintaining high specificity, which makes it suitable for practical screening of potentially interesting high-temperature superconductors.

The feature-selection models from Scenario 3 provided useful insight into predictor redundancy and interpretability. However, they did not outperform Random Forest. Backward stepwise logistic regression achieved strong performance but retained many predictors, which limited interpretability. Lasso with `lambda.1se` provided the best compromise among the feature-selection methods, because it retained fewer predictors while preserving similar performance, but the model was still not highly sparse.

Therefore, the final recommended model for prediction is **Random Forest with the Youden threshold**. It provides the best practical trade-off between detecting `high_tc` materials and limiting false positive predictions.

## Conclusion

This project investigated whether high-temperature superconductors can be identified from numerical material descriptors. The original continuous target `critical_temp` was transformed into a binary classification problem using the 77 K threshold. The exploratory analysis showed that several predictors separate the two classes, but also revealed substantial redundancy and strong correlations among many descriptors.

In Scenario 1, multiple classification models were compared. Random Forest achieved the best overall predictive performance and was selected as the final recommended model. It outperformed logistic regression, SVM and decision tree models, suggesting that non-linear relationships and interactions between descriptors are important for this classification task.

In Scenario 3, feature-selection methods were evaluated to study whether the predictor space could be reduced while preserving performance. Backward stepwise logistic regression, lasso and elastic net all retained a relatively large number of predictors. This indicates that the predictive information is not concentrated in only a small subset of variables, but is distributed across many material descriptors. Among the feature-selection methods, lasso with `lambda.1se` provided the best compromise between model compactness and predictive performance.

Overall, the results support the hypothesis that numerical material descriptors contain useful information for distinguishing `high_tc` and `non_high_tc` materials. Random Forest is the most suitable model for predictive performance, while regularized logistic regression is useful for understanding feature-selection behaviour and predictor stability.

## Project Submission Contents

The project submission contains all files required to reproduce the analysis, inspect the results, and run the interactive Shiny application.

The submitted files include:

- **R Markdown documentation** (`Superconductivity_Classification_Project.Rmd`) containing the full technical report, including data description, EDA, preprocessing, modelling, evaluation, feature selection, final model selection and reproducibility information.
- **Rendered HTML documentation** generated from the R Markdown file.
- **One-page executive summary** summarizing the project goal, main findings and final recommendation in non-technical language.
- **Shiny application** for interactive exploration of the dataset and comparison of selected classification models.
- **Original dataset files**, including the original superconductivity data used in the project.
- **Prepared data file** (`prepared_data.RData` or equivalent), created during preprocessing and used to simplify loading in later scripts and in the Shiny application.

- **R scripts**, including the EDA script (00\_eda.R) and additional modelling or preparation scripts used to generate intermediate outputs.
- **Fitted model objects**, stored separately to make the Shiny application load faster without retraining all models every time.
- **Feature-selection output** (task3\_feature\_retention.csv), containing the retained features and comparison of feature-selection methods.

## Reproducibility

To improve reproducibility, fixed random seeds were used before train/test splitting and cross-validation. This ensures that the same data split and the same cross-validation folds can be reused when the analysis is rerun. The following session information documents the R version and package versions used to generate this report.

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Slovak_Slovakia.utf8  LC_CTYPE=Slovak_Slovakia.utf8
## [3] LC_MONETARY=Slovak_Slovakia.utf8 LC_NUMERIC=C
## [5] LC_TIME=Slovak_Slovakia.utf8
##
## time zone: Europe/Bratislava
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] magrittr_2.0.4      pROC_1.19.0.1      caret_7.0-1
## [4] lattice_0.22-7      MASS_7.3-65         glmnet_4.1-10
## [7] Matrix_1.7-4        pheatmap_1.0.13    patchwork_1.3.2
## [10] lubridate_1.9.5     forcats_1.0.1      stringr_1.6.0
## [13] dplyr_1.2.0         purrr_1.2.1        readr_2.2.0
## [16] tidyr_1.3.2         tibble_3.3.1       ggplot2_4.0.2
## [19] tidyverse_2.0.0     MLmetrics_1.1.3    rpart.plot_3.1.4
## [22] rpart_4.1.24        randomForest_4.7-1.2 e1071_1.7-17
##
## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.1    timeDate_4052.112  farver_2.1.2
## [4] S7_0.2.1            fastmap_1.2.0      digest_0.6.39
## [7] timechange_0.4.0    lifecycle_1.0.5    survival_3.8-3
## [10] compiler_4.5.2      rlang_1.1.7        tools_4.5.2
## [13] utf8_1.2.6          yaml_2.3.12        data.table_1.18.2.1
## [16] knitr_1.51          labeling_0.4.3     bit_4.6.0
## [19] plyr_1.8.9          RColorBrewer_1.1-3 withr_3.0.2
## [22] nnet_7.3-20         grid_4.5.2         stats4_4.5.2
## [25] future_1.70.0       globals_0.19.1     scales_1.4.0
```

|                           |                      |                  |
|---------------------------|----------------------|------------------|
| ## [28] iterators_1.0.14  | cli_3.6.5            | crayon_1.5.3     |
| ## [31] rmarkdown_2.30    | generics_0.1.4       | otel_0.2.0       |
| ## [34] rstudioapi_0.18.0 | future.apply_1.20.2  | reshape2_1.4.5   |
| ## [37] tzdb_0.5.0        | proxy_0.4-29         | splines_4.5.2    |
| ## [40] parallel_4.5.2    | vctrs_0.7.1          | hardhat_1.4.3    |
| ## [43] hms_1.1.4         | bit64_4.6.0-1        | listenv_0.10.1   |
| ## [46] foreach_1.5.2     | gower_1.0.2          | recipes_1.3.2    |
| ## [49] glue_1.8.0        | parallelly_1.47.0    | codetools_0.2-20 |
| ## [52] stringi_1.8.7     | shape_1.4.6.1        | gtable_0.3.6     |
| ## [55] pillar_1.11.1     | htmltools_0.5.9      | ipred_0.9-15     |
| ## [58] lava_1.9.0        | R6_2.6.1             | vroom_1.7.0      |
| ## [61] evaluate_1.0.5    | class_7.3-23         | Rcpp_1.1.1-1.1   |
| ## [64] nlme_3.1-168      | proclim_2026.03.11   | xfun_0.56        |
| ## [67] pkgconfig_2.0.3   | ModelMetrics_1.2.2.2 |                  |